MODUL PRAKTIKUM MINGGU 9

ARSITEKTUR WEB SERVICES TERDISTRIBUSI, IMPLEMENTASI RINGKAS LAYANAN SOAP (XML ENVELOPE, KONTRAK WSDL, & SOAP FAULT), SERTA PERANCANGAN & IMPLEMENTASI MENDALAM RESTFUL WEB API (SERVICE CONSUMER, SERVICE PROVIDER, SEMANTIK HTTP, ELOQUENT RESOURCE, HATEOAS, SANCTUM BEARER TOKEN, RFC 7807, & OPENAPI 3.0) PADA LARAVEL 11.X

Mata Kuliah	: Pemrograman dan Pengujian Aplikasi Web (12S3101) — SKS: 4 (3-1)
Keterkaitan Materi	: Melanjutkan HTTP (W1), Session (W4), MVC (W5), & Database/ACID (W6) menuju komunikasi antar-mesin (M2M); fondasi langsung Minggu 10–13
Dosen Pengampu	: Chandro Pardede, S.Kom., M.Sc.
Alokasi Waktu	: 170 Menit (Praktikum Terbimbing: Lab 1 SOAP Ringkas + Lab 2–5 REST API Mendalam) + 360 Menit (Tugas Mandiri Proyek)
Format Penugasan	: Repositori Publik GitHub (`ppw-2026-week9-[NIM]` atau branch `week9-rest-api`) — Melanjutkan Proyek Mandiri `SIPUS-Del` Minggu 6

I. CAPAIAN PEMBELAJARAN & TUJUAN PRAKTIKUM

1.1 Capaian Pembelajaran (Sub-CPMK)

> **SubCPMK12-02-09-07: Mahasiswa mampu menganalisis kebutuhan dan feasibility penerapan web service mengembangkan dan menggunakan web service berbasis REST dengan tepat. [C4, A3, P3]**

1.2 Indikator Ketercapaian

Setelah menyelesaikan seluruh rangkaian kegiatan praktikum pada modul ini, mahasiswa diharapkan mampu:

1. **Menganalisis Kebutuhan & Kelayakan (*Feasibility*) Web Services**: Membedakan aplikasi web berorientasi manusia (*HTML Rendering*) dengan layanan web berorientasi mesin (*XML/JSON Data Exchange*), serta memetakan kapan suatu sistem berperan sebagai **Service Consumer** maupun **Service Provider**.
2. **Memahami & Mengimplementasikan Layanan SOAP Secara Ringkas**: Membangun simulasi ringkas layanan berbasis protokol **SOAP (*Simple Object Access Protocol*)** pada Laravel 11 untuk memahami struktur `<soap:Envelope>`, `<soap:Header>`, `<soap:Body>`, penanganan galat `<soap:Fault>`, serta kontrak **WSDL (*Web Services Description Language*)** pada integrasi sistem *legacy/enterprise*.
3. **Menguasai Prinsip Arsitektur REST & Segitiga REST**: Menjelaskan 6 batasan arsitektural Roy Fielding (khususnya sifat **Statelessness** dan **Uniform Interface**), segitiga elemen REST (*Nouns*, *Verbs*, *Representations*), serta tingkatan kematangan **Richardson Maturity Model (Level 0 s.d. Level 3 HATEOAS)**.
4. **Mengonsumsi REST API Publik & Mengintegrasikan Laravel HTTP Client**: Membaca kontrak dokumentasi **GitHub REST API**, menginspeksi *Raw HTTP Headers* melalui **cURL** dan **Postman**, menggunakan *Personal Access Token (PAT)* pada operasi mutasi `PATCH /user`, serta memanggil API eksternal dari dalam kode Laravel menggunakan fasad `Http`.
5. **Membangun Arsitektur RESTful Provider yang Bersih di Laravel 11**: Mengonfigurasi jalur nir-status `routes/api.php` melalui `php artisan install:api`, memisahkan struktur tabel internal database dari representasi JSON publik menggunakan **Eloquent API Resource (`JsonResource`)**, serta menyusun amplop respons JSON yang konsisten.

6. **Menerapkan Semantik HTTP Verbs & Status Codes Secara Presisi**: Mengimplementasikan operasi `GET` (`200 OK`), `POST` (`201 Created` beserta header `Location`), membedakan secara nyata perilaku `PUT` (*Full Replacement*) dengan `PATCH` (*Partial Update*), serta `DELETE` (`204 No Content` dan penolakan relasi `409 Conflict`).
7. **Mengintegrasikan Logika Transaksi Bisnis ACID ke dalam Endpoint REST**: Mengekspos proses bisnis kritis (pendaftaran KRS / peminjaman buku) menggunakan `DB::transaction()` dan `lockForUpdate()` melalui endpoint RESTful yang aman dari *race condition*.
8. **Menerapkan Otentikasi Stateless Berbasis Bearer Token**: Melindungi endpoint mutasi data menggunakan **Laravel Sanctum Bearer Token** (`Authorization: Bearer <token>`), menguji penerbitan token saat login API, serta pencabutan token (*token revocation*) saat logout tanpa menyimpan sesi di memori server.
9. **Menstandarkan Penanganan Galat API (RFC 7807) & Hypermedia (HATEOAS)**: Mengonfigurasi exception handler global pada `bootstrap/app.php` agar seluruh galat (`401`, `404`, `409`, `422`, `500`) dikembalikan dalam struktur JSON standar **RFC 7807 (*Problem Details*)**, serta menyertakan navigasi tautan mandiri `_links`.
10. **Mendokumentasikan Kontrak Layanan & Menguji Endpoint Secara Sistematis**: Menyusun dokumen kontrak **OpenAPI 3.0 (`openapi.yaml`)** dan mengorganisasi pengujian skenario positif maupun negatif menggunakan *Postman Collection & Environment*.

1.3 Peta Alokasi Waktu Praktikum Terbimbing (170 Menit)

Sesi Lab	Fokus Kegiatan Hands-On	Durasi	Peran Arsitektur
LAB 1	Implementasi Ringkas Layanan SOAP: Kontrak WSDL, `<soap:Envelope>`, & `<soap:Fault>`	25 Menit	SOAP Provider & Consumer
LAB 2	Eksplorasi GitHub REST API (cURL & Postman), Bearer PAT, `PATCH`, & Laravel `Http` Client	35 Menit	REST Service Consumer
LAB 3	Arsitektur `routes/api.php`, `ApiResponse` Trait, `CourseResource`, HATEOAS, & Filtering/Pagination	40 Menit	REST Provider (Read Layer)
LAB 4	Mutasi Semantik (`POST`, `PUT` vs `PATCH`, `DELETE`), Transaksi KRS ACID, & Auth Token Sanctum	45 Menit	REST Provider (Write & Auth)
LAB 5	Kontrak OpenAPI 3.0 (`openapi.yaml`), Matriks Uji Postman Collection, & Troubleshooting	25 Menit	Contract & API Verification

II. ALAT DAN BAHAN (PREREQUISITES)

Sebelum memulai sesi praktikum, pastikan seluruh perangkat lunak berikut telah siap pada komputer Anda:

Komponen Perangkat	Spesifikasi Minimum	Perintah Verifikasi / Keterangan
1. Basis Kode Minggu 6	Proyek `sim-del` & `SIPUS-Del`	Proyek terbimbing (`sim-del`) untuk sesi Lab 1–5, dan proyek mandiri (`SIPUS-Del`) untuk Tugas Mandiri Minggu 9. *(Tersedia panduan Quick-Setup jika Anda memulai dari folder baru).*
2. PHP Runtime CLI	PHP 8.2 atau PHP 8.3	Jalankan `php -v` dan `php -m`. Pastikan ekstensi `curl`, `json`, `xml`, `dom`, `simplexml`, `mbstring`, `openssl`, dan `pdo_sqlite`/`pdo_mysql` aktif.
3. Composer & Laravel	Composer 2.6+ & Laravel 11.x	Jalankan `composer --version` dan `php artisan --version`.
4. API Testing Client	**Postman Desktop** (atau Thunder Client / Bruno / SoapUI)	Digunakan untuk merakit HTTP Method (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`), mengatur Header XML/JSON, menginspeksi Raw HTTP Response, dan mengelola Bearer Token.

Komponen Perangkat	Spesifikasi Minimum	Perintah Verifikasi / Keterangan
5. Terminal & cURL CLI	PowerShell / Git Bash / Terminal	Jalankan `curl.exe --version` (pada PowerShell Windows, gunakan `curl.exe` agar tidak tertukar dengan alias `Invoke-WebRequest`).
6. Akun GitHub Aktif	Akun pada `https://github.com`	Digunakan pada Lab 2 untuk menguji konsumsi REST API publik dan membuat *Personal Access Token (PAT)*.

> **TIPS KHUSUS PENGGUNA WINDOWS POWERSHELL**:
> Di Windows PowerShell, perintah `curl` secara bawaan adalah *alias* dari cmdlet `Invoke-WebRequest` yang memiliki sintaks berbeda dari cURL standar! Oleh karena itu, jika Anda menggunakan PowerShell, **selalu ketikkan `curl.exe`** (dengan akhiran `.exe`), atau gunakan terminal **Git Bash / Command Prompt (CMD)** agar seluruh flag `-i`, `-X`, `-H`, dan `-d` berjalan 100% akurat.

III. LANDASAN TEORI DAN PENJELASAN MENDALAM

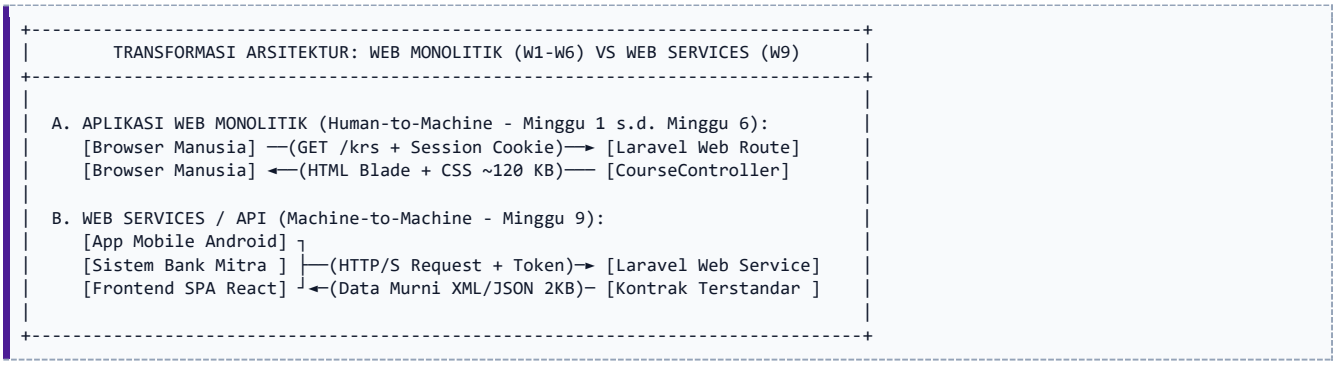
3.1 Motivasi Web Services: Prinsip *Domain Focus*, Orkestrasi, & *Service Contract*

Pada paruh pertama perkuliahan (Minggu 1 hingga Minggu 6), kita membangun aplikasi web **monolitik berorientasi manusia**. Pengguna membuka peramban (*browser*), mengirim *HTTP Request*, lalu Laravel mengeksekusi kueri database dan mengembalikan halaman visual **HTML + CSS (Blade)** seberat puluhan hingga ratusan kilobyte untuk ditampilkan di layar.

Namun, di dunia industri modern berlaku prinsip **"No Super-Enterprise, No Super-App"**: tidak ada satu aplikasi pun yang sanggup dan layak membangun seluruh kapabilitas bisnis sendirian dari nol.

- Sistem Informasi Akademik (**SIAKAD IT Del**) harus fokus mengelola kurikulum, KRS, dan nilai mahasiswa — bukan membangun sistem pencetak *Virtual Account* perbankan atau server peta satelit sendiri.
- Ketika aplikasi **GoFood** atau **Tokopedia** memproses satu pesanan pelanggan dalam hitungan detik, aplikasi tersebut melakukan **Orkestrasi Layanan (*Service Orchestration*)**: merangkai komunikasi ke *Merchant Service* (cek stok), *Promo Service* (hitung diskon), *Payment Gateway* (potong saldo), *Maps/Dispatch Service* (cari kurir terdekat), dan *Notification Service* (kirim pesan ke ponsel).

Agar aplikasi yang dibangun dengan bahasa pemrograman berbeda (misalnya SIAKAD berbasis **PHP Laravel**, aplikasi Mobile Mahasiswa berbasis **Flutter/Kotlin**, dan sistem Bank berbasis **Java Enterprise**) dapat saling berbicara melalui jaringan internet tanpa antarmuka manusia, dibutuhkan **Web Service**.



Konsep Kunci yang Wajib Dipahami:

1. **Service & Web Service**: *Service* adalah fungsionalitas perangkat lunak dengan tujuan bisnis spesifik yang diekspos melalui **API (*Application Programming Interface*)**. Ketika service tersebut diakses melalui protokol web standar (**HTTP/HTTPS** pada port 80/443), ia disebut **Web Service**.

2. **Transparent Business Logic (Abstraksi Kotak Hitam / *Black Box*)**: Konsumen layanan tidak perlu tahu (dan tidak peduli!) bahasa pemrograman apa, tabel database apa, atau berapa banyak sub-service yang dipanggil di balik layar oleh penyedia layanan. Selama input yang dikirim sesuai dan output yang diterima benar, logika internal sepenuhnya terenkapsulasi.
3. **Service Contract (Kontrak Layanan)**: Kesepakatan resmi yang mengikat antara penyedia dan pengguna layanan. Setiap kontrak API wajib memuat minimal **3 spesifikasi**:
   - **Cara Mengonsumsi (*How to Consume*)**: Alamat URL/Endpoint, protokol, metode pemanggilan, dan skema autentikasi.
   - **Spesifikasi Parameter (*Input Parameters*)**: Nama parameter, letaknya (di URL path, query string, header, atau body), tipe data, serta sifat wajib/opsional.
   - **Ekspektasi Luaran (*Expected Output*)**: Struktur data kembalian (XML/JSON) beserta kode status keberhasilan maupun kegagalan.
4. **Dua Peran dalam Ekosistem Layanan (*Consumer* vs *Provider*)**:
   - Anda bertindak sebagai **Service Consumer** ketika aplikasi Anda *memanggil* API milik pihak lain (karena data/fitur tersebut berada di luar domain bisnis inti Anda, dikelola resmi oleh instansi lain, atau jauh lebih hemat biaya dibanding membuat sendiri — contoh: memanggil GitHub API, Google OAuth, atau API Kurs Bank Indonesia).
   - Anda bertindak sebagai **Service Provider** ketika aplikasi Anda *membuka dan menyediakan* endpoint API agar data/kapabilitas yang menjadi domain sah milik Anda dapat diakses secara terkontrol oleh aplikasi lain (contoh: `sim-del` menyediakan API data mata kuliah bagi aplikasi Mobile Mahasiswa).

3.2 Mengenal Arsitektur SOAP (*Simple Object Access Protocol*) Secara Ringkas

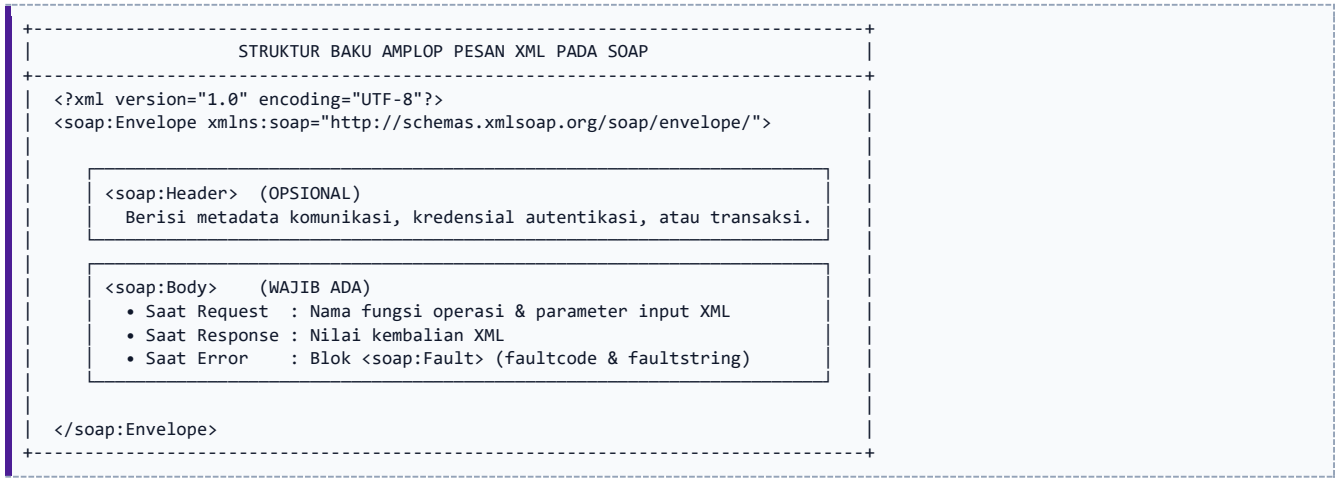
Sebelum REST mendominasi pengembangan web dan mobile modern, standar utama pertukaran layanan enterprise adalah **SOAP**, yang distandarisasi secara ketat oleh konsorsium **W3C**. Berbeda dengan REST yang merupakan *gaya arsitektur*, **SOAP adalah protokol komunikasi formal** yang mewajibkan seluruh pesan dikirim dalam format **XML (*eXtensible Markup Language*)** dengan struktur baku.

Tiga Pilar Dasar Arsitektur SOAP:

1. **SOAP (*Simple Object Access Protocol*)**: Aturan struktur amplop pesan XML yang dipertukarkan saat *request* dan *response*.
2. **WSDL (*Web Services Description Language*)**: Dokumen kontrak formal berformat XML (`.wsdl`) yang mendeskripsikan daftar nama operasi (*method*), tipe parameter XSD, nilai balik, dan URL layanan. File WSDL bersifat **wajib (*mandatory*)** pada SOAP tradisional agar *tool* di sisi klien dapat men-generate kelas perantara otomatis (*Client Stub*).
3. **UDDI (*Universal Description, Discovery, and Integration*)**: Direktori pencatatan layanan (ibarat buku telepon *Yellow Pages*) tempat *Provider* mempublikasikan WSDL (*Publish*) dan *Consumer* mencari layanan (*Find*) sebelum mengikat koneksi (*Bind*). *(Selain ketiganya, di level perbankan/korporasi besar terdapat spesifikasi lanjutan **WS-*** seperti **WS-Security** untuk enkripsi & tanda tangan digital level dokumen XML).*

Anatomi Amplop Pesan SOAP (`<soap:Envelope>`) & `<soap:Fault>`:

Setiap pesan SOAP (baik permintaan maupun balasan) wajib dibungkus di dalam satu elemen akar `<soap:Envelope>` yang menggunakan *XML Namespace* (`xmlns:soap`) agar tag protokol tidak pernah bentrok dengan tag data bisnis aplikasi:



Kapan SOAP Masih Digunakan Saat Ini?

Karena ukuran dokumen XML sangat besar (*verbose*), boros *bandwidth*, proses *parsing* memakan CPU lebih tinggi, dan hanya memanfaatkan HTTP `POST` sebagai pipa saluran (*tunneling*), SOAP jarang dipilih untuk proyek startup, web, atau mobile baru. Namun, sebagai sarjana Sistem Informasi, Anda **tetap wajib memahami dasar SOAP** karena sistem inti (*core system*) di industri **perbankan, asuransi, pajak/pemerintahan, dan telekomunikasi** banyak yang masih mengoperasikan layanan SOAP warisan (*legacy*) yang harus Anda integrasikan dengan aplikasi REST baru!

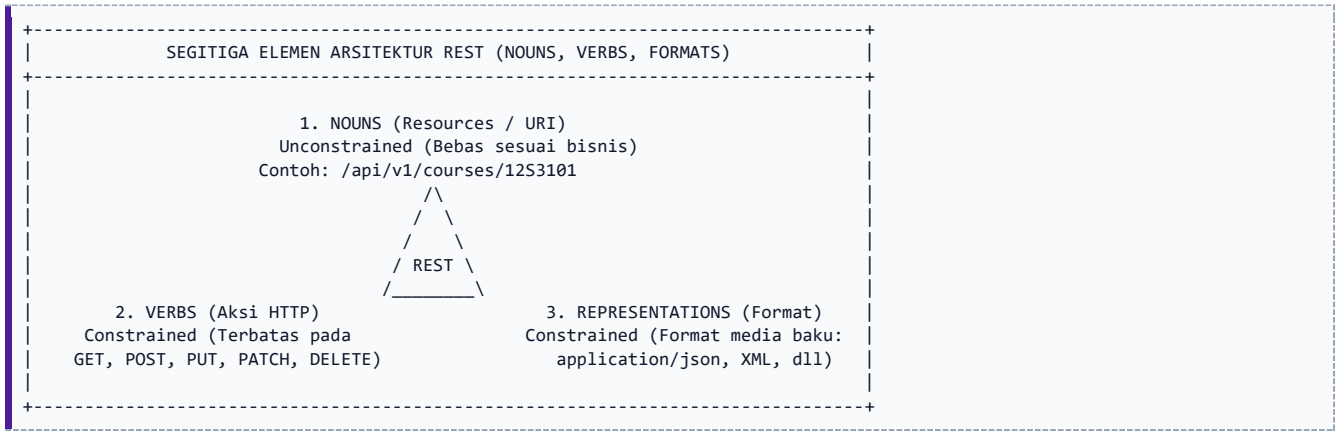
3.3 Mengupas Tuntas Arsitektur REST (*Representational State Transfer*)

Diperkenalkan oleh **Roy Thomas Fielding** (salah satu perancang utama spesifikasi protokol HTTP/1.1) dalam disertasi doktoralnya pada tahun 2000, **REST** bukanlah protokol baru dan bukan pula pustaka kode, melainkan **pola/gaya arsitektur (*architectural style*)** yang memanfaatkan seluruh kehebatan alami protokol **HTTP** apa adanya tanpa lapisan pembungkus XML yang rumit.

> **MEMAHAMI MAKNA KATA "REPRESENTATIONAL STATE TRANSFER"**:
> Bayangkan di dalam tabel `courses` pada database server Anda terdapat baris data mata kuliah `12S3101` dengan sisa kuota `45` kursi. Nilai `45` di tabel database tersebut adalah **State (kondisi terkini sumber daya)**. Ketika aplikasi ponsel mahasiswa meminta data mata kuliah itu, server **tidak mengirimkan tabel fisik SQLite/MySQL-nya**, melainkan mengubah *state* tersebut menjadi format teks standar (seperti **JSON**) lalu mentransfernya melalui HTTP ke ponsel klien. Pertukaran "*representasi dari state sumber daya*" inilah asal-usul nama **Representational State Transfer (REST)**!

A. Segitiga Elemen REST (*The REST Triangle*)

Setiap layanan berbasis REST bertumpu pada tiga elemen utama:



1. **Nouns (Kata Benda / *Resources*)**: Segala entitas bisnis yang dapat diberi nama dan dialamatkan melalui URI (*Uniform Resource Identifier*), seperti `/courses`, `/students`, `/enrollments`. Bersifat *unconstrained* karena setiap organisasi bebas menamai entitas bisnisnya.
2. **Verbs (Kata Kerja / *HTTP Methods*)**: Aksi yang dilakukan terhadap resource tersebut. Bersifat *constrained* (dibatasi ketat hanya menggunakan metode resmi protokol HTTP: `GET`, `POST`, `PUT`, `PATCH`, `DELETE`), bukan mengarang kata kerja sendiri di dalam URL!
3. **Representations (Format Data)**: Wujud pengemasan data yang dipertukarkan antara server dan klien. Saat ini **JSON (`application/json`)** menjadi standar *de facto* karena ukurannya hingga 5–8 kali lebih ringkas dibanding XML SOAP dan langsung didukung (*native*) oleh JavaScript, PHP, Python, Dart, Kotlin, maupun Go.

B. Enam Batasan Arsitektural REST (Roy Fielding) & Mengapa *Statelessness* Sangat Penting

Agar sebuah API layak disebut *RESTful*, ia dirancang mengikuti 6 batasan arsitektur:

1. **Client-Server Separation**: Antarmuka pengguna (Frontend/Mobile) terpisah penuh dari penyimpanan data & logika bisnis (Backend). Keduanya dapat dikembangkan oleh tim berbeda secara paralel.
2. **Statelessness (Nir-Status — SANGAT KRUSIAL!)**:
   - Bandingkan dengan **Minggu 6 (Stateful Session)**: Saat user login di web biasa, Laravel menyimpan data sesi user di server (file/tabel `sessions`). Jika ada 100.000 pengguna aktif, server harus terus mengingat 100.000 sesi tersebut. Jika kita memasang 3 server di belakang *Load Balancer*, pengguna yang masuk ke Server #1 pada request pertama bisa ditolak oleh Server #2 pada request kedua karena file sesinya tertinggal di Server #1!
   - Pada **REST (Stateless)**: Server **TIDAK menyimpan sesi klien sama sekali (*Zero Server Session*)**. Setiap HTTP Request yang masuk wajib berdiri mandiri (*self-contained*), membawa seluruh parameter dan bukti identitasnya sendiri (berupa **Access Token** pada header `Authorization: Bearer ...`). Hasilnya: request ke-1 boleh dilayani Server #1, request ke-2 dilayani Server #2, dan request ke-3 dilayani Server #3 dengan mulus!
3. **Cacheability**: Respons server memberi tahu klien/proxy apakah data boleh disimpan sementara (*cache*) melalui header seperti `Cache-Control` dan `ETag` sehingga menghemat kuota jaringan.
4. **Uniform Interface**: Keseragaman cara mengakses resource di seluruh endpoint (identifikasi URI konsisten, manipulasi via representasi, pesan yang menjelaskan dirinya sendiri via Header HTTP, serta *Hypermedia/HATEOAS*).
5. **Layered System**: Klien tidak perlu tahu apakah ia terhubung langsung ke server Laravel utama atau melewati lapisan *API Gateway*, *Cloudflare WAF*, dan *Load Balancer*.
6. **Code-on-Demand (Opsional)**: Kemampuan server mengirimkan skrip eksekusi ke klien bila diperlukan.

C. Tangga Kematangan *Richardson Maturity Model (RMM)*

Bagaimana cara menilai apakah API yang kita bangun sudah benar-benar menerapkan prinsip REST atau sebenarnya masih bergaya RPC/SOAP yang hanya diganti formatnya ke JSON? Leonard Richardson membaginya menjadi 4 level:

Level RMM	Nama Tingkatan	Ciri Khas Implementasi	Contoh Nyata
Level 0	*The Swamp of POX* (RPC bergaya JSON)	Satu URL sapu jagat dan hanya memakai metode `POST` untuk semua aksi (baca, tambah, hapus).	`POST /api/akademik` dengan body `{"action": "getCourse", "id": 1}`
Level 1	*Resources*	Sudah memecah alamat URL per entitas bisnis, tetapi masih menyalahgunakan metode HTTP atau menaruh kata kerja di URL.	`POST /api/courses/1/update` atau `GET /api/courses/1/delete`
Level 2	**HTTP Verbs & Status Codes** *(Standar Minimal Industri)*	Menggunakan kata benda pada URI + memadukannya dengan **HTTP Verbs** (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`) + **HTTP Status Codes** semantik (`200`, `201`, `204`, `404`, `422`).	`DELETE /api/v1/courses/1` ⟶ merespons `204 No Content`
Level 3	**Hypermedia Controls (HATEOAS)** *(Puncak REST)*	Memenuhi Level 2, ditambah setiap objek JSON menyertakan panduan tautan aksi berikutnya (`_links`) secara dinamis.	JSON mata kuliah memuat `"_links": {"enrollments": "/api/v1/courses/1/enrollments"}`

3.4 Semantik Protokol HTTP: Matriks Verbs, *PUT vs PATCH*, & Status Codes

Keunggulan utama REST terletak pada **Semantik**: hanya dengan melirik baris pertama HTTP Request dan HTTP Response, seorang engineer langsung memahami niat operasi dan hasil akhirnya tanpa perlu membongkar isi dokumen!

A. Matriks HTTP Verbs, *Safety*, dan *Idempotency*

Dua sifat teknis yang wajib Anda pahami saat merancang endpoint API:

- **Safe (Aman)**: Pemanggilan endpoint **tidak mengubah data apa pun** di server. Dipanggil 1.000 kali pun isi database tetap sama.
- **Idempotent (Idempoten)**: Efek perubahan di database jika request yang persis sama dieksekusi **1 kali** adalah **identik** dengan jika request tersebut tidak sengaja terkirim ulang **5 kali** (misalnya akibat sinyal ponsel pengguna berkedip lalu aplikasi melakukan *auto-retry*).

HTTP Verb	Contoh Endpoint	Tujuan Bisnis (CRUD)	Safe?	Idempotent?	Mengapa Idempotent / Tidak?
`GET`	`GET /api/v1/courses`	Membaca daftar/detail resource	**Ya**	**Ya**	Hanya membaca data; tidak ada mutasi state di server.
`POST`	`POST /api/v1/courses`	Membuat resource baru / transaksi baru	Tidak	**Tidak**	Dikirim 5x beruntun dapat menciptakan 5 baris data baru (atau memotong kuota 5x)!
`PUT`	`PUT /api/v1/courses/7`	Mengganti **seluruh** isi resource ID 7	Tidak	**Ya**	Menimpa baris ID 7 dengan objek `X` sebanyak 1x maupun 5x hasil akhirnya tetap objek `X`.
`PATCH`	`PATCH /api/v1/courses/7`	Mengubah **sebagian** field resource ID 7	Tidak	*Kondisional*	Idempoten jika mengisi nilai tetap (`{"quota": 50}`), non-idempoten jika bersifat inkremental.
`DELETE`	`DELETE /api/v1/courses/7`	Menghapus resource ID 7	Tidak	**Ya**	Request ke-1 menghapus baris ID 7; request ke-2 s.d. ke-5 mendapati baris ID 7 tetap sudah terhapus.

B. Ilustrasi Konkret Perbedaan `PUT` vs `PATCH` (Sering Keluar di Ujian & Wawancara Kerja!)

Misalkan di database saat ini tersimpan mata kuliah dengan ID `1`:

JSON

```
{ "id": 1, "code": "12S3101", "name": "PPW", "credits": 4, "quota": 40, "description": "Wajib Semester 5" }
```

Sekarang dosen ingin mengubah **`quota` saja menjadi `60`**:

- Jika menggunakan **`PUT /api/v1/courses/1`**, klien **WAJIB mengirimkan ulang seluruh atribut lengkap** (`code`, `name`, `credits`, `quota: 60`, `description`). Jika klien lupa mengirim `description`, maka pada semantik `PUT` murni, kolom `description` akan dianggap dikosongkan (`null`)!
- Jika menggunakan **`PATCH /api/v1/courses/1`**, klien **CUKUP mengirimkan `{"quota": 60}`** saja! Atribut `code`, `name`, `credits`, dan `description` tidak tersentuh dan tetap aman di database. Itulah sebabnya pada **GitHub API** (Slide Minggu 9), operasi mengubah `bio` profil menggunakan **`PATCH /user`**, bukan `PUT /user`!

C. Standar Pemilihan HTTP Response Status Codes pada REST API

> ⚠ **LARANGAN KERAS ANTI-PATTERN REST**:
> Jangan pernah mengirimkan status HTTP **`200 OK`** ketika terjadi kegagalan, misalnya mereturn status `200` dengan isi JSON `{"status": false, "message": "Data tidak ditemukan"}`! Hal tersebut merusak cara kerja *HTTP Client* (Axios, Fetch, Postman, API Gateway). Gunakan kode status HTTP yang jujur sesuai kategorinya:

Kode Status	Nama Standar	Kapan Wajib Digunakan pada REST API Anda?
`200 OK`	Sukses Umum	Berhasil mengambil data (`GET`) atau berhasil memperbarui data (`PUT` / `PATCH`) yang mengembalikan body JSON.
`201 Created`	Berhasil Dibuat	Berhasil menciptakan baris data baru melalui `POST`. Biasakan menyertakan header `Location: /api/v1/resources/{id}`.
`204 No Content`	Sukses Tanpa Isi	Berhasil menghapus data melalui `DELETE` di mana server tidak perlu mengirimkan body JSON apa pun.
`400 Bad Request`	Permintaan Rusak	Klien mengirim sintaks JSON yang cacat (kurung kurawal kurang) atau tipe parameter query tidak masuk akal.
`401 Unauthorized`	Belum Terautentikasi	Klien belum melampirkan *Bearer Token*, atau token yang dikirim salah / sudah kedaluwarsa (*expired* / *revoked*).
`403 Forbidden`	Akses Ditolak	Identitas token **valid** (server tahu siapa Anda), tetapi hak akses/peran Anda tidak diizinkan menyentuh resource ini.
`404 Not Found`	Tidak Ditemukan	ID resource yang diminta pada URL (misal `/api/v1/courses/9999`) tidak ada di database, atau URL salah ketik.
`405 Method Not Allowed`	Metode Terlarang	URL benar, tetapi HTTP Verb salah (misal mengirim `DELETE` ke endpoint yang hanya mengizinkan `GET`).
`409 Conflict`	Konflik State Bisnis	Permintaan berbenturan dengan kondisi data saat ini (contoh: NIM mendaftar ulang ke MK yang sama, atau menghapus MK yang sudah punya peserta KRS).
`422 Unprocessable Content`	Gagal Validasi Input	Sintaks JSON valid, tetapi isinya melanggar aturan validasi bisnis (contoh: SKS diisi `-3`, kode MK kosong, atau kuota penuh).
`429 Too Many Requests`	Melampaui Batas Kuota	Klien memanggil API terlalu cepat melebihi batas *Rate Limiting* (`throttle:api`).
`500 Internal Server Error`	Kegagalan Server	Terjadi bug kode PHP atau koneksi database putus di sisi server (bukan salah input dari klien).

3.5 Kaidah Desain URI Bersih, Standar Error RFC 7807, & Kontrak OpenAPI 3.0

A. Enam Kaidah Emas Merancang URI RESTful

1. **Gunakan Kata Benda Jamak (*Plural Nouns*), Hindari Kata Kerja**: Gunakan `/api/v1/courses`, **bukan** `/api/v1/getCourses`, `/api/v1/createCourse`, atau `/api/v1/deleteCourseById`.
2. **Gunakan Hierarki Path untuk Relasi Kepemilikan (*Sub-Resource*)**: Untuk mengambil daftar mahasiswa yang mengambil mata kuliah ID `5`, gunakan `GET /api/v1/courses/5/enrollments`.
3. **Gunakan Huruf Kecil (*Lowercase*) dan Tanda Hubung (*Kebab-Case*)**: Gunakan `/api/v1/academic-years`, hindari huruf besar/camelCase (`/academicYears`) atau garis bawah (`/academic_years`).
4. **Gunakan Query Parameters untuk Filter, Pencarian, Urutan, & Paginasi**: Contoh: `GET /api/v1/courses?semester=Genap&credits=4&sort=code&per_page=10`.
5. **Sertakan Versi API Sejak Awal (`/api/v1/...`)**: Jika tahun depan struktur JSON berubah drastis, Anda cukup merilis `/api/v2/...` tanpa mematikan aplikasi mobile lama mahasiswa yang masih memanggil `/api/v1/...`.
6. **Hindari Garis Miring Ganda atau di Ujung (*No Trailing Slash*)**: Gunakan `/api/v1/courses`, bukan `/api/v1/courses/`.

B. Konsistensi Amplop JSON, RFC 7807 Problem Details, & Kontrak OpenAPI

- Pada SOAP, kontrak didefinisikan secara kaku dalam file `.wsdl`. Pada REST, kontrak ditulis dalam bentuk **Dokumentasi API**. Agar dokumentasi REST tidak mudah usang atau berbeda dengan kode program, industri modern menggunakan spesifikasi **OpenAPI 3.0 (Swagger)** dalam file `.yaml` atau `.json`.
- Untuk respons galat (*error*), standar IETF **RFC 7807 (*Problem Details for HTTP APIs*)** menganjurkan pengembalian objek JSON yang seragam memuat atribut `type`, `title`, `status`, `detail`, dan `instance` (beserta daftar `errors` per kolom jika

statusnya `422`), sehingga pengembang Frontend/Mobile dapat menampilkan pesan kesalahan dengan sangat mudah dan terprediksi.

C. Tabel Sintesis: Web Monolitik (W6) vs SOAP Service vs RESTful API

Aspek Pembanding	Web Monolitik (Minggu 6)	SOAP Web Service (Minggu 9)	RESTful Web API (Minggu 9)
Target Konsumen	Manusia via Web Browser	Mesin-ke-Mesin (Sistem Legacy)	Mesin-ke-Mesin (Mobile, SPA, Mitra)
Lokasi Rute Laravel	`routes/web.php`	Custom Endpoint XML POST	`routes/api.php` (Prefix `/api`)
Format Payload	HTML + CSS (`Blade View`)	Wajib XML (`<soap:Envelope>`)	Fleksibel, dominan **JSON**
Manajemen State	**Stateful** (`Session` + Cookie)	Umumnya Stateless (bisa WS-Session)	**Wajib Stateless** (Bearer Token)
Proteksi CSRF Cookie	Wajib Aktif (`@csrf`)	Tidak relevan (Non-Browser)	Non-aktif (Diganti Token Auth)
Kontrak Layanan	Tidak ada (Antarmuka Visual)	Wajib File **WSDL** (`.wsdl`)	Dokumentasi API / **OpenAPI 3.0**

IV. KEGIATAN PRAKTIKUM TERBIMBING (HANDS-ON LABS)

Persiapan Awal Lingkungan Proyek `sim-del` (5 Menit Sebelum Lab Dimulai)

Praktikum Minggu 9 ini melanjutkan proyek **`sim-del`** dari Minggu 6 yang telah memiliki tabel **`courses`** (mata kuliah), **`enrollments`** (pendaftaran KRS), dan **`users`**. Buka terminal pada folder proyek `sim-del` Anda:

BASH

```
cd sim-del
php artisan migrate:status
```

> **PANDUAN CEPAT (JIKA ANDA BELUM MEMILIKI PROYEK `sim-del` DARI MINGGU 6)**:
> Jika karena suatu hal laptop Anda baru atau proyek Minggu 6 tidak terbawa, Anda tetap dapat mengikuti seluruh modul ini dari nol hanya dalam 2 menit! Jalankan perintah berikut:
>
> 1. `composer create-project laravel/laravel:^11.0 sim-del && cd sim-del`
> 2. `php artisan make:model Course -m && php artisan make:model Enrollment -m`
> 3. Pada migration `courses`, isi kolom: `$table->string('code', 10)->unique(); $table->string('name', 150); $table->unsignedTinyInteger('credits'); $table->string('semester', 30); $table->unsignedInteger('quota')->default(40); $table->text('description')->nullable();`
> 4. Pada migration `enrollments`, isi kolom: `$table->foreignId('course_id')->constrained('courses')->cascadeOnDelete(); $table->string('student_nim', 15); $table->string('student_name', 150); $table->string('semester', 30); $table->unique(['course_id', 'student_nim']);`
> 5. Pada `app/Models/Course.php`, tambahkan `protected $fillable = ['code','name','credits','semester','quota','description'];` dan relasi `public function enrollments() { return $this->hasMany(Enrollment::class); }`.
> 6. Pada `app/Models/Enrollment.php`, tambahkan `protected $fillable = ['course_id','student_nim','student_name','semester'];` dan relasi `public function course() { return $this->belongsTo(Course::class); }`.
> 7. Buat minimal 1 akun user dan 3 sampel mata kuliah di `DatabaseSeeder.php` (misal email: `akademik@del.ac.id`, password: `password123`), lalu jalankan `php artisan migrate --seed`.

Pastikan server lokal menyala di satu tab terminal tersendiri selama praktikum berlangsung:

BASH

```
php artisan serve
```

LAB 1: Implementasi Ringkas Layanan SOAP — Kontrak WSDL, `<soap:Envelope>`, & `<soap:Fault>` (25 Menit)

> **Skenario Studi Kasus Lab 1**:
> Bank Mitra Kampus yang menangani tagihan uang kuliah mahasiswa IT Del masih menggunakan sistem inti (*Core Banking*) warisan berbasis **SOAP**. Sistem Bank membutuhkan sebuah endpoint SOAP pada `sim-del` bernama operasi **`GetStudentSks`** untuk mengecek total SKS yang diambil seorang mahasiswa berdasarkan `nim`. Mari kita bangun dan uji layanan SOAP ringkas ini agar Anda memahami cara kerja WSDL, SOAP Envelope, dan SOAP Fault secara nyata!

Langkah 1.1: Membuat `AcademicSoapController`

Buat controller baru melalui terminal kedua:

BASH

```
php artisan make:controller Soap/AcademicSoapController
```

Buka berkas **`app/Http/Controllers/Soap/AcademicSoapController.php`**, lalu tuliskan kode lengkap yang ringkas dan bersih berikut:

PHP

```
<?php

namespace App\Http\Controllers\Soap;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use Illuminate\Http\Response;
use Illuminate\Support\Facades\DB;

class AcademicSoapController extends Controller
{
    /**
     * 1. Menyajikan dokumen Kontrak WSDL (GET /soap/academic.wsdl)
     */
    public function wsdl(): Response
    {
        $endpointUrl = url('/soap/academic');

        $wsdlXml = <<<XML
<?xml version="1.0" encoding="UTF-8"?>
<definitions name="AcademicSoapService"
             targetNamespace="http://sim.del.ac.id/soap/academic"
             xmlns:tns="http://sim.del.ac.id/soap/academic"
             xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
             xmlns:xsd="http://www.w3.org/2001/XMLSchema"
             xmlns="http://schemas.xmlsoap.org/wsdl/">

    <!-- 1. Message: Spesifikasi parameter input dan output -->
    <message name="GetStudentSksRequest">
        <part name="nim" type="xsd:string"/>
    </message>
    <message name="GetStudentSksResponse">
        <part name="nim" type="xsd:string"/>
        <part name="student_name" type="xsd:string"/>
        <part name="total_courses" type="xsd:int"/>
        <part name="total_sks" type="xsd:int"/>
    </message>

    <!-- 2. PortType: Daftar operasi (method) yang disediakan -->
    <portType name="AcademicPortType">
        <operation name="GetStudentSks">
            <input message="tns:GetStudentSksRequest"/>
            <output message="tns:GetStudentSksResponse"/>
        </operation>
    </portType>

    <!-- 3. Binding: Mengikat operasi ke protokol SOAP over HTTP -->
```

```
    <binding name="AcademicSoapBinding" type="tns:AcademicPortType">
        <soap:binding style="rpc" transport="http://schemas.xmlsoap.org/soap/http"/>
        <operation name="GetStudentSks">
            <soap:operation soapAction="http://sim.del.ac.id/soap/academic#GetStudentSks"/>
            <input><soap:body use="literal" namespace="http://sim.del.ac.id/soap/academic"/></input>
            <output><soap:body use="literal" namespace="http://sim.del.ac.id/soap/academic"/></output>
        </operation>
    </binding>

    <!-- 4. Service: Alamat URL fisik tempat endpoint SOAP berada -->
    <service name="AcademicSoapService">
        <port name="AcademicPort" binding="tns:AcademicSoapBinding">
            <soap:address location="{$endpointUrl}"/>
        </port>
    </service>
</definitions>
XML;

        return response($wsdlXml, 200)
            ->header('Content-Type', 'text/xml; charset=utf-8');
    }

    /**
     * 2. Memproses Pesan Masuk SOAP Envelope (POST /soap/academic)
     */
    public function handle(Request $request): Response
    {
        $rawXml = trim($request->getContent());

        if (empty($rawXml)) {
            return $this->soapFault('soap:Client', 'Body XML SOAP tidak boleh kosong.');
        }

        // Parsing dokumen XML dengan aman (tanpa bergantung ekstensi ext-soap eksternal)
        libxml_use_internal_errors(true);
        $xml = simplexml_load_string($rawXml);

        if ($xml === false) {
            return $this->soapFault('soap:Client', 'Format XML tidak valid (Malformed XML).');
        }

        // Ambil nilai tag <nim> di dalam <soap:Body> terlepas dari prefix namespace-nya
        $nimNodes = $xml->xpath('//*[local-name()="GetStudentSksRequest"]/*[local-name()="nim"]');

        if (empty($nimNodes)) {
            return $this->soapFault(
                'soap:Client',
                'Operasi tidak dikenali atau elemen <nim> tidak ditemukan pada GetStudentSksRequest.'
            );
        }

        $nim = trim((string) $nimNodes[0]);

        // Hitung data KRS mahasiswa dari tabel enrollments & courses (Materi Minggu 6)
        $rekap = DB::table('enrollments')
            ->join('courses', 'enrollments.course_id', '=', 'courses.id')
            ->where('enrollments.student_nim', $nim)
            ->selectRaw('MAX(enrollments.student_name) as student_name, COUNT(courses.id) as total_mk,
COALESCE(SUM(courses.credits), 0) as total_sks')
            ->first();

        if (!$rekap || $rekap->total_mk == 0) {
            return $this->soapFault(
                'soap:Client',
                "Data pengambilan KRS untuk NIM '{$nim}' tidak ditemukan di sistem akademik."
            );
        }

        $safeNim  = htmlspecialchars($nim, ENT_XML1, 'UTF-8');
        $safeName = htmlspecialchars((string) $rekap->student_name, ENT_XML1, 'UTF-8');

        // Bungkus hasil di dalam amplop resmi <soap:Envelope> & <soap:Body>
        $responseXml = <<<XML
<?xml version="1.0" encoding="UTF-8"?>
```

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:m="http://sim.del.ac.id/soap/academic">
    <soap:Header>
        <m:ProviderInfo>SIM-Del Legacy SOAP Engine v1.0</m:ProviderInfo>
    </soap:Header>
    <soap:Body>
        <m:GetStudentSksResponse>
            <m:nim>{$safeNim}</m:nim>
            <m:student_name>{$safeName}</m:student_name>
            <m:total_courses>{$rekap->total_mk}</m:total_courses>
            <m:total_sks>{$rekap->total_sks}</m:total_sks>
        </m:GetStudentSksResponse>
    </soap:Body>
</soap:Envelope>
XML;

        return response($responseXml, 200)
            ->header('Content-Type', 'text/xml; charset=utf-8');
    }

    /**
     * Helper pembentuk pesan kesalahan standar protokol SOAP (<soap:Fault>)
     */
    private function soapFault(string $faultCode, string $faultMessage): Response
    {
        $safeMsg = htmlspecialchars($faultMessage, ENT_XML1, 'UTF-8');

        $faultXml = <<<XML
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
    <soap:Body>
        <soap:Fault>
            <faultcode>{$faultCode}</faultcode>
            <faultstring>{$safeMsg}</faultstring>
        </soap:Fault>
    </soap:Body>
</soap:Envelope>
XML;

        return response($faultXml, 500)
            ->header('Content-Type', 'text/xml; charset=utf-8');
    }
}
```

BEDAH KODE SINGKAT LAB 1:

- Method **`wsdl()`** mempublikasikan kontrak formal WSDL yang berisi 4 bagian inti: `<message>` (tipe parameter), `<portType>` (nama fungsi `GetStudentSks`), `<binding>` (aturan SOAP HTTP), dan `<service>` (URL tujuan).
- Method **`handle()`** bertindak sebagai *SOAP Engine* ringkas: membaca *raw body* XML via `$request->getContent()`, mengekstrak `<nim>` menggunakan XPath bebas-namespace, menjalankan kueri `JOIN` ke tabel `enrollments` & `courses` yang kita bangun di Minggu 6, lalu membungkus hasilnya ke dalam `<soap:Envelope>`.
- Method **`soapFault()`** menunjukkan karakteristik unik SOAP: ketika terjadi error, SOAP wajib mengirimkan blok **`<soap:Fault>`** di dalam `<soap:Body>`.

Langkah 1.2: Mendaftarkan Rute SOAP & Mengecualikan CSRF Web

1. Tambahkan 2 baris rute berikut di bagian bawah berkas **`routes/web.php`**:

PHP

```
use App\Http\Controllers\Soap\AcademicSoapController;

Route::get('/soap/academic.wsdl', [AcademicSoapController::class, 'wsdl']);
Route::post('/soap/academic', [AcademicSoapController::class, 'handle']);
```

2. Karena rute di dalam `routes/web.php` secara otomatis dijaga oleh middleware `ValidateCsrfToken` (yang mengharapkan input `@csrf` dari form HTML browser), sedangkan pemanggil SOAP adalah mesin eksternal, buka **`bootstrap/app.php`** dan kecualikan URI `/soap/*` pada blok `withMiddleware`:

PHP

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->validateCsrfTokens(except: [
        'soap/*',
    ]);
})
```

Langkah 1.3: Menguji Kontrak WSDL, Request SOAP Sukses, & `<soap:Fault>`

1. **Inspeksi WSDL di Browser**: Buka `http://localhost:8000/soap/academic.wsdl` pada browser Chrome/Edge. Anda akan melihat struktur pohon kontrak XML WSDL tampil rapi!
2. **Pastikan Terdapat Minimal 1 Data KRS**: Jika tabel `enrollments` Anda masih kosong, masukkan 1 data uji cepat melalui Tinker:

BASH

```
php artisan tinker --execute="App\Models\Enrollment::firstOrCreate(['course_id' => App\Models\Course::first()->id,
'student_nim' => '12S22001'], ['student_name' => 'Bona Pandiangan', 'semester' => '2025/2026-Genap']);"
```

3. **Uji Kirim Pesan `<soap:Envelope>` via Postman (atau cURL)**:
   - Di **Postman**: Buat request baru, ubah metode menjadi **`POST`**, isi URL `http://localhost:8000/soap/academic`.
   - Buka tab **Headers**, tambahkan `Content-Type` bernilai `text/xml; charset=utf-8`.
   - Buka tab **Body** -> pilih **raw** -> pilih tipe **XML**, lalu tempelkan amplop SOAP berikut dan klik **Send**:

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
               xmlns:m="http://sim.del.ac.id/soap/academic">
    <soap:Body>
        <m:GetStudentSksRequest>
            <m:nim>12S22001</m:nim>
        </m:GetStudentSksRequest>
    </soap:Body>
</soap:Envelope>
```

   - Amati balasan XML yang diterima! Terlihat jelas `<soap:Header>` dan `<soap:Body>` yang memuat `<m:total_sks>`.
4. **Uji Skenario `<soap:Fault>`**: Ubah nilai `<m:nim>` menjadi `99S99999` (NIM tidak terdaftar) lalu klik **Send** kembali. Amati bagaimana server mengembalikan blok `<soap:Fault>` berisi `<faultcode>soap:Client</faultcode>` dan penjelasan kesalahannya!

> **CATATAN TAMBAHAN (KONSUMSI DENGAN PHP NATIVE `SoapClient`)**:
> Di dunia kerja nanti, jika ekstensi `extension=soap` pada `php.ini` diaktifkan, seorang *Service Consumer* di PHP tidak perlu merakit string XML di atas secara manual, melainkan cukup memanggil kontrak WSDL menggunakan 2 baris kode *Stub* otomatis:
> `$client = new \SoapClient('http://localhost:8000/soap/academic.wsdl');`
> `$hasil  = $client->GetStudentSks('12S22001');`
> Namun perhatikan: hanya untuk mengirim 8 karakter NIM (`12S22001`), jaringan tetap harus mentransfer ratusan byte tag XML `<soap:Envelope>` menggunakan metode `POST`! Sekarang, mari kita buktikan betapa ringkas, ekspresif, dan elegannya **REST API** pada Lab 2 s.d. Lab 5!

LAB 2: Menjadi *REST Service Consumer* — Eksplorasi Mendalam GitHub REST API & Laravel HTTP Client (35 Menit)

Sesuai materi perkuliahan Minggu 9 (Modul `09-03`), cara terbaik menguasai REST sebelum membangun API sendiri adalah dengan membedah dan mengonsumsi salah satu REST API terbaik di dunia industri: **GitHub REST API (`https://api.github.com`)**.

Langkah 2.1: Membaca Kontrak Dokumentasi GitHub REST API

Sebelum mengetik satu baris perintah pun, seorang engineer wajib membaca kontrak dokumentasi resmi di `https://docs.github.com/en/rest/users/users`. Untuk operasi **"Get a user"**, kita mengekstrak 3 elemen kontrak:

1. **Cara Mengonsumsi**: HTTP Verb = `GET`, Endpoint = `https://api.github.com/users/{username}`.
2. **Spesifikasi Parameter**:
   - Header `Accept` = `application/vnd.github+json` *(memberi tahu server bahwa klien meminta representasi JSON versi v3 standar GitHub)*.
   - Path Parameter `{username}` = handle akun GitHub yang dicari (contoh: `octocat` atau username GitHub pribadi Anda).
3. **Ekspektasi Output**: Kode status `200 OK` dengan body JSON profil publik pengguna (atau `404 Not Found` jika username tidak ada).

Langkah 2.2: Konsumsi Publik via cURL & Inspeksi Mendalam HTTP Headers

Jalankan perintah berikut di terminal (ganti `octocat` dengan username GitHub Anda sendiri jika punya!):

BASH

```
curl.exe -i -H "Accept: application/vnd.github+json" https://api.github.com/users/octocat
```

Amati bagian atas luaran terminal (**Raw HTTP Response Headers**):

HTTP

```
HTTP/2 200
date: Tue, 03 Nov 2026 02:15:00 GMT
content-type: application/json; charset=utf-8
cache-control: public, max-age=60, s-maxage=60
etag: W/"a1b2c3d4..."
x-github-media-type: github.v3; format=json
x-ratelimit-limit: 60
x-ratelimit-remaining: 59
x-ratelimit-used: 1
```

Mari kita pahami makna setiap baris header tersebut:

- `HTTP/2 200`: Semantik keberhasilan permintaan.
- `content-type: application/json; charset=utf-8`: Representasi state yang dikirim adalah objek JSON murni (hanya beberapa ratus byte, bandingkan dengan XML SOAP!).
- `cache-control: public, max-age=60`: Bukti batasan **Cacheability** REST — respons ini boleh disimpan di cache selama 60 detik tanpa membebani server GitHub lagi.
- `x-ratelimit-limit: 60` & `x-ratelimit-remaining: 59`: Karena kita memanggil **tanpa token identitas**, GitHub membatasi alamat IP kita maksimal **60 request per jam**.

Sekarang bukalah alamat `https://api.github.com/users/octocat` langsung di *address bar* browser Anda. Mengapa browser dapat menampilkannya secara langsung? **Karena operasi ini menggunakan metode `GET`!** Namun ingat: *address bar* browser biasa tidak bisa mengirimkan metode `POST`, `PATCH`, `DELETE`, maupun menyisipkan header `Authorization` kustom — itulah alasan kita menggunakan **Postman**!

Langkah 2.3: Menguji Query Parameters di Postman

1. Buka **Postman**, buat request baru dengan metode `GET`.
2. Masukkan URL koleksi repositori milik `octocat`: `https://api.github.com/users/octocat/repos?sort=updated&direction=desc&per_page=3`

3. Perhatikan tab **Params** di Postman: Postman otomatis memecah parameter `sort`, `direction`, dan `per_page` menjadi tabel *Key-Value* yang mudah diedit.
4. Klik **Send**. Anda hanya menerima tepat **3 objek repositori terbaru** milik `octocat` dalam array JSON `[ {...}, {...}, {...} ]`!

Langkah 2.4: Otentikasi Stateless (*Personal Access Token*) & Operasi `PATCH /user`

Bagaimana jika Anda ingin **mengubah biodata (`bio`)** pada akun GitHub Anda sendiri melalui REST API? Operasi ini mengubah data privat, sehingga wajib terotorisasi. Karena REST bersifat **Stateless** (tidak ada form login session), GitHub menggunakan **Token**.

1. **Membuat Personal Access Token (PAT) di GitHub**:
   - Login ke `https://github.com` -> klik foto profil di kanan atas -> **Settings**.
   - Gulir ke paling bawah di menu kiri -> klik **Developer settings** -> **Personal access tokens** -> **Tokens (classic)** -> **Generate new token (classic)**.
   - Isi *Note*: `Praktikum-PPW-Week9`, pilih *Expiration*: `7 days`, lalu centang kotak scope `user` (*Update ALL user data*).
   - Klik **Generate token** dan segera salin string token yang diawali `ghp_...` tersebut ke tempat catatan sementara di laptop Anda.

> ⚠ **ATURAN KEAMANAN MUTLAK — DO NOT SHARE YOUR TOKEN!**:
> Token `ghp_...` memiliki kekuasaan setara password akun GitHub Anda sesuai *scope* yang dicentang! **Dilarang keras** menempelkan token asli ke dalam kode yang di-commit ke GitHub. Saat mengambil *screenshot* laporan praktikum, sensor sebagian karakter token Anda, dan hapus (*Delete/Revoke*) token tersebut setelah praktikum selesai.

2. **Menguji `PATCH https://api.github.com/user` di Postman**:
   - Berdasarkan kontrak GitHub "*Update the authenticated user*", metode yang dipakai adalah `PATCH` (karena kita hanya ingin mengubah 1 atribut yaitu `bio`, tanpa menghapus `name`, `email`, atau `blog`!).
   - Buat tab baru di Postman: pilih metode `PATCH` dan URL `https://api.github.com/user`.
   - Buka tab **Authorization** -> pilih Type: **Bearer Token** -> tempelkan token `ghp_...` Anda pada kolom **Token**. *(Di balik layar, Postman otomatis menyisipkan header `Authorization: Bearer ghp_...` pada request Anda!)*
   - Buka tab **Headers** -> pastikan ada `Accept` = `application/vnd.github+json`.
   - Buka tab **Body** -> pilih **raw** -> pilih tipe **JSON**, lalu ketikkan:

JSON

```
{
  "bio": "Mahasiswa S1 Sistem Informasi IT Del — Praktikum REST API Minggu 9"
}
```

   - Klik **Send**!
   - **Amati 3 Bukti Keberhasilan**:
     1. Status respons menunjukkan `200 OK` dan di dalam JSON balasan nilai `"bio"` sudah berubah!
     2. Buka tab **Headers** pada bagian Response Postman: lihat `x-ratelimit-limit` kini melonjak dari `60` menjadi `5000` request/jam karena request bersifat *authenticated*!
     3. Refresh halaman profil GitHub Anda di web browser (`https://github.com/username_anda`): tulisan bio profil Anda benar-benar telah berubah!
   - **Uji Negatif (`401 Unauthorized`)**: Hapus 2 huruf terakhir pada Bearer Token di Postman lalu klik **Send**. Amati status `401 Unauthorized` dengan body `{"message": "Bad credentials"}`!

Langkah 2.5: Mengonsumsi REST API Eksternal dari Dalam Kode Laravel 11 (`Http` Facade)

Pada Langkah 2.1–2.4 kita menjadi *Consumer* menggunakan alat manual (cURL & Postman). Bagaimana caranya agar **aplikasi Laravel `sim-del` kita** yang bertindak sebagai *Service Consumer* secara otomatis memanggil API GitHub? Laravel 11 menyediakan pembungkus Guzzle yang sangat ekspresif bernama `Illuminate\Support\Facades\Http`!

1. Buat controller baru:

BASH

```
php artisan make:controller Api/V1/GithubPortfolioController
```

2. Buka `app/Http/Controllers/Api/V1/GithubPortfolioController.php` dan isi dengan kode berikut:

PHP

```
<?php

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Http;

class GithubPortfolioController extends Controller
{
    /**
     * Laravel bertindak sebagai SERVICE CONSUMER yang memanggil GitHub REST API
     * Endpoint: GET /api/v1/portfolios/github/{username}
     */
    public function show(string $username): JsonResponse
    {
        // 1. Siapkan HTTP Client dengan Header kontrak resmi & batas waktu 10 detik
        $client = Http::accept('application/vnd.github+json')
            ->withHeaders(['X-GitHub-Api-Version' => '2022-11-28'])
            ->timeout(10);

        // Opsional: Jika GITHUB_TOKEN diisi di .env, lampirkan sebagai Bearer Token
        if ($token = env('GITHUB_TOKEN')) {
            $client = $client->withToken($token);
        }

        // 2. Panggil endpoint profil & repositori dari GitHub API
        $userRes = $client->get("https://api.github.com/users/{$username}");

        // 3. Tangani jika user tidak ditemukan (404) atau GitHub sedang gangguan
        if ($userRes->status() === 404) {
            return response()->json([
                'success' => false,
                'message' => "Akun GitHub '{$username}' tidak ditemukan.",
            ], 404);
        }

        if ($userRes->failed()) {
            return response()->json([
                'success' => false,
                'message' => 'Gagal berkomunikasi dengan layanan eksternal GitHub API.',
            ], 502); // 502 Bad Gateway: Server upstream/eksternal bermasalah
        }

        $reposRes = $client->get("https://api.github.com/users/{$username}/repos", [
            'sort'     => 'updated',
            'per_page' => 3,
        ]);

        $profil = $userRes->json();
        $repos  = collect($reposRes->json())->map(fn ($r) => [
            'repo_name'   => $r['name'],
            'language'    => $r['language'],
            'stars'       => $r['stargazers_count'],
            'html_url'    => $r['html_url'],
        ]);

        // 4. Kembalikan hasil orkestrasi yang sudah diringkas sesuai kebutuhan SIM-Del
        return response()->json([
            'success' => true,
            'message' => 'Data portofolio koding mahasiswa berhasil diambil dari GitHub API.',
            'data'    => [
                'github_login' => $profil['login'],
                'display_name' => $profil['name'] ?? $profil['login'],
                'bio'          => $profil['bio'],
                'public_repos' => $profil['public_repos'],
```

```
                'followers'    => $profil['followers'],
                'profile_url'  => $profil['html_url'],
                'latest_repos' => $repos,
            ],
        ], 200);
    }
}
```

> 💡 **CATATAN**: Rute untuk `GithubPortfolioController` ini akan langsung kita daftarkan bersama seluruh rute `routes/api.php` pada langkah pertama **Lab 3** di bawah ini!

LAB 3: Menjadi *REST Service Provider* (Bagian 1) — Arsitektur `routes/api.php`, `CourseResource`, HATEOAS, & Pencarian Terpaginasi (40 Menit)

Mulai Lab 3 ini, kita beralih peran menjadi **Service Provider**: membangun layanan RESTful API resmi milik `sim-del` agar data akademik dapat diakses oleh aplikasi Mobile Mahasiswa dan sistem mitra kampus secara aman, terstruktur, dan mudah dipahami!

Langkah 3.1: Mengaktifkan Infrastruktur API di Laravel 11 (`install:api`)

Secara bawaan, instalasi awal Laravel 11 hanya menyertakan `routes/web.php` agar ringan. Untuk mengaktifkan kanal REST API resmi, jalankan perintah Artisan berikut pada terminal proyek `sim-del`:

BASH

```
php artisan install:api
```

*(Jika muncul pertanyaan `One new database migration has been published. Would you like to run all pending database migrations?`, ketik `yes` lalu tekan **Enter**).*

Apa saja yang otomatis dikerjakan oleh perintah `php artisan install:api` tersebut?

1. Mengunduh dan mengonfigurasi paket resmi `laravel/sanctum` untuk otentikasi token nir-status (*stateless API token*).
2. Membuat tabel migrasi `personal_access_tokens` di database untuk mencatat token API yang diterbitkan.
3. Membuat berkas rute baru `routes/api.php`.
4. Mendaftarkan jalur `api: __DIR__.'/../routes/api.php'` di dalam `bootstrap/app.php`, sehingga seluruh rute di dalam `routes/api.php` otomatis:
   - Memiliki awalan URL `/api/...` (contoh: rute `/v1/courses` menjadi `http://localhost:8000/api/v1/courses`).
   - Berjalan di bawah grup middleware `api` yang **stateless** (tanpa cookie session browser dan bebas dari pengecekan form `@csrf`).

> ⚠ **JEBAKAN PEMULA NOMOR #1 DI LARAVEL API (`Accept: application/json`)**:
> Saat menguji endpoint Laravel menggunakan Postman atau cURL, **SELALU tambahkan Request Header `Accept: application/json`**! Mengapa? Jika Anda mengirim input yang salah tanpa header `Accept: application/json`, Laravel mengira Anda adalah pengguna browser web biasa sehingga ia melakukan *redirect* `302 Found` ke halaman utama HTML, bukan mengembalikan pesan validasi JSON `422`!

Langkah 3.2: Membangun Trait Standarisasi Respons JSON & RFC 7807 (`ApiResponse`)

Agar seluruh controller API kita memiliki struktur output JSON yang **100% seragam, rapi, dan tidak mengulang-ulang kode**, mari kita buat sebuah *Trait* pembantu. Buat folder `app/Traits` (jika belum ada) dan buat berkas `app/Traits/ApiResponse.php`:

PHP

```
<?php

namespace App\Traits;
```

```php
use Illuminate\Http\JsonResponse;

trait ApiResponse
{
    /**
     * Respons sukses standar (Default: 200 OK)
     */
    protected function successResponse(
        mixed $data = null,
        string $message = 'Permintaan berhasil diproses.',
        int $status = 200,
        array $meta = []
    ): JsonResponse {
        $payload = [
            'success' => true,
            'message' => $message,
            'data'    => $data,
        ];

        if (!empty($meta)) {
            $payload['meta'] = $meta;
        }

        return response()->json($payload, $status);
    }

    /**
     * Respons sukses pembuatan resource baru (201 Created + Header Location)
     */
    protected function createdResponse(
        mixed $data,
        string $locationUrl,
        string $message = 'Resource baru berhasil dibuat.'
    ): JsonResponse {
        return $this->successResponse($data, $message, 201)
            ->header('Location', $locationUrl);
    }

    /**
     * Respons sukses hapus data tanpa body (204 No Content)
     */
    protected function noContentResponse(): JsonResponse
    {
        return response()->json(null, 204);
    }

    /**
     * Respons galat berstandar IETF RFC 7807 (Problem Details for HTTP APIs)
     */
    protected function problemResponse(
        string $title,
        string $detail,
        int $status,
        string $errorSlug = 'general-error',
        array $errors = []
    ): JsonResponse {
        $problem = [
            'type'     => url("/docs/errors/{$errorSlug}"),
            'title'    => $title,
            'status'   => $status,
            'detail'   => $detail,
            'instance' => request()->getRequestUri(),
        ];

        if (!empty($errors)) {
            $problem['errors'] = $errors;
        }

        return response()
            ->json($problem, $status)
            ->header('Content-Type', 'application/problem+json');
    }
}
```

Langkah 3.3: Membuat Eloquent API Resource (`EnrollmentResource` & `CourseResource`) + HATEOAS

Mengapa pengembang profesional **tidak langsung menulis** `return Course::all();` di Controller?

1. Jika nama kolom di tabel database berubah suatu hari nanti, aplikasi Mobile klien akan langsung rusak (*tight coupling*).
2. Kolom internal sensitif dapat tidak sengaja ikut terekspos.
3. Kita tidak leluasa menyisipkan atribut terhitung (seperti `remaining_quota`) maupun tautan navigasi **HATEOAS (`_links`)**.

Solusi resminya di Laravel adalah **Eloquent API Resource**. Jalankan perintah berikut di terminal:

BASH

```
php artisan make:resource Api/V1/EnrollmentResource
php artisan make:resource Api/V1/CourseResource
```

1. Buka `app/Http/Resources/Api/V1/EnrollmentResource.php` dan sesuaikan isinya:

PHP

```
<?php

namespace App\Http\Resources\Api\V1;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class EnrollmentResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'           => $this->id,
            'student_nim'  => $this->student_nim,
            'student_name' => $this->student_name,
            'semester'     => $this->semester,
            'enrolled_at'  => $this->created_at?->toIso8601String(),
            '_links'       => [
                'cancel_enrollment' => [
                    'href'   => url("/api/v1/enrollments/{$this->id}"),
                    'method' => 'DELETE',
                ],
            ],
        ];
    }
}
```

2. Buka `app/Http/Resources/Api/V1/CourseResource.php` dan tuliskan transformasi berikut:

PHP

```
<?php

namespace App\Http\Resources\Api\V1;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class CourseResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        // Hitung jumlah pendaftar jika withCount('enrollments') dipanggil
        $enrolledCount = $this->whenCounted('enrollments', fn () => $this->enrollments_count, 0);
        $isFull        = $this->quota <= 0;

        // Bangun kontrol Hypermedia HATEOAS (Richardson Maturity Model Level 3)
        $links = [
            'self' => [
                'href'   => url("/api/v1/courses/{$this->id}"),
                'method' => 'GET',
            ],
```

```
            'enrollments' => [
                'href'   => url("/api/v1/courses/{$this->id}/enrollments"),
                'method' => 'GET',
            ],
            'update_full' => [
                'href'   => url("/api/v1/courses/{$this->id}"),
                'method' => 'PUT',
            ],
            'update_partial' => [
                'href'   => url("/api/v1/courses/{$this->id}"),
                'method' => 'PATCH',
            ],
            'delete' => [
                'href'   => url("/api/v1/courses/{$this->id}"),
                'method' => 'DELETE',
            ],
        ];

        // Keindahan HATEOAS: Link aksi 'enroll_student' HANYA muncul jika kuota masih tersedia!
        if (!$isFull) {
            $links['enroll_student'] = [
                'href'   => url("/api/v1/courses/{$this->id}/enrollments"),
                'method' => 'POST',
            ];
        }

        return [
            'id'             => $this->id,
            'code'           => $this->code,
            'name'           => $this->name,
            'credits'        => (int) $this->credits,
            'semester'       => $this->semester,
            'remaining_quota'=> (int) $this->quota,
            'enrolled_count' => $enrolledCount,
            'status_label'   => $isFull ? 'FULL' : 'OPEN',
            'description'    => $this->description,
            // Relasi daftar peserta HANYA dimuat jika controller memanggil ->load('enrollments')
            'students'       => EnrollmentResource::collection($this->whenLoaded('enrollments')),
            'updated_at'     => $this->updated_at?->toIso8601String(),
            '_links'         => $links,
        ];
    }
}
```

> **PAHAMI CARA KERJA HATEOAS DI ATAS**:
> Perhatikan blok `if (!$isFull)`! Ketika sisa kuota mata kuliah masih `> 0`, JSON menyertakan tautan `_links.enroll_student`. Begitu kuota habis (`0`), tautan `_links.enroll_student` **otomatis menghilang** dari respons JSON! Aplikasi Mobile/Frontend cukup memeriksa keberadaan link tersebut untuk menyalakan atau mematikan tombol *"Ambil KRS"* tanpa perlu menduplikasi rumus bisnis backend di sisi klien!

Langkah 3.4: Membangun Operasi Baca (`index` & `show`) pada `CourseApiController`

Buat controller API khusus versi 1:

BASH

```
php artisan make:controller Api/V1/CourseApiController
```

Buka **`app/Http/Controllers/Api/V1/CourseApiController.php`** dan tuliskan kode lengkap untuk operasi baca (`index` dan `show`) terlebih dahulu:

PHP

```
<?php

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
```

```php
use App\Http\Resources\Api\V1\CourseResource;
use App\Models\Course;
use App\Traits\ApiResponse;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;

class CourseApiController extends Controller
{
    use ApiResponse;

    /**
     * GET /api/v1/courses
     * Mengambil koleksi mata kuliah dengan Filter, Search, Sorting, & Paginasi
     */
    public function index(Request $request): JsonResponse
    {
        // 1. Batasi parameter pengurutan pada kolom yang diizinkan (mencegah SQL Injection pada ORDER BY)
        $allowedSorts = ['code', 'name', 'credits', 'quota', 'created_at'];
        $sortBy       = in_array($request->query('sort'), $allowedSorts) ? $request->query('sort') : 'code';
        $sortDir      = strtolower($request->query('dir', 'asc')) === 'desc' ? 'desc' : 'asc';
        $perPage      = min(max((int) $request->query('per_page', 5), 1), 50);

        // 2. Susun kueri Eloquent yang efisien dengan withCount('enrollments')
        $paginator = Course::query()
            ->withCount('enrollments')
            ->when($request->query('search'), function ($q, $search) {
                $q->where(function ($sub) use ($search) {
                    $sub->where('code', 'like', "%{$search}%")
                        ->orWhere('name', 'like', "%{$search}%");
                });
            })
            ->when($request->query('credits'), fn ($q, $sks) => $q->where('credits', (int) $sks))
            ->when($request->query('semester'), fn ($q, $sem) => $q->where('semester', $sem))
            ->when($request->boolean('available'), fn ($q) => $q->where('quota', '>', 0))
            ->orderBy($sortBy, $sortDir)
            ->paginate($perPage)
            ->withQueryString();

        // 3. Kembalikan koleksi terbungkus CourseResource beserta metadata paginasi
        return $this->successResponse(
            data: CourseResource::collection($paginator->items()),
            message: 'Daftar koleksi mata kuliah berhasil diambil.',
            meta: [
                'current_page' => $paginator->currentPage(),
                'per_page'     => $paginator->perPage(),
                'total_items'  => $paginator->total(),
                'total_pages'  => $paginator->lastPage(),
                'links'        => [
                    'first' => $paginator->url(1),
                    'prev'  => $paginator->previousPageUrl(),
                    'next'  => $paginator->nextPageUrl(),
                    'last'  => $paginator->url($paginator->lastPage()),
                ],
            ]
        );
    }

    /**
     * GET /api/v1/courses/{id}
     * Mengambil detail satu mata kuliah berdasarkan ID atau Kode MK (contoh: /courses/1 atau /courses/12S3101)
     */
    public function show(string $identifier): JsonResponse
    {
        $course = Course::withCount('enrollments')
            ->with('enrollments')
            ->where('id', $identifier)
            ->orWhere('code', strtoupper($identifier))
            ->first();

        if (!$course) {
            return $this->problemResponse(
                title: 'Resource Not Found',
                detail: "Mata kuliah dengan pengenal '{$identifier}' tidak ditemukan di basis data.",
                status: 404,
```

```
                errorSlug: 'course-not-found'
            );
        }

        return $this->successResponse(
            data: new CourseResource($course),
            message: 'Detail mata kuliah berhasil ditemukan.'
        );
    }
}
```

Langkah 3.5: Uji Cepat Endpoint Baca (`GET`)

Tambahkan sementara rute baca di `routes/api.php` untuk langsung menguji hasil Lab 2 & Lab 3 Anda:

PHP

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\V1\CourseApiController;
use App\Http\Controllers\Api\V1\GithubPortfolioController;

Route::prefix('v1')->group(function () {
    Route::get('/portfolios/github/{username}', [GithubPortfolioController::class, 'show']);
    Route::get('/courses', [CourseApiController::class, 'index']);
    Route::get('/courses/{identifier}', [CourseApiController::class, 'show']);
});
```

Uji di Postman (atau terminal cURL) dengan 4 variasi berikut dan perhatikan betapa bersihnya JSON yang dihasilkan:

1. `GET http://localhost:8000/api/v1/courses?per_page=3&sort=quota&dir=desc` -> Status **`200 OK`** + blok `meta` paginasi!
2. `GET http://localhost:8000/api/v1/courses/1` (atau gunakan kode MK seperti `/api/v1/courses/12S3101`) -> Status **`200 OK`** lengkap dengan array `students` dan `_links`!
3. `GET http://localhost:8000/api/v1/courses/MK-TIDAK-ADA` -> Status **`404 Not Found`** dengan header `Content-Type: application/problem+json`!
4. `GET http://localhost:8000/api/v1/portfolios/github/octocat` -> Status **`200 OK`** hasil orkestrasi konsumsi GitHub API!

LAB 4: Menjadi *REST Service Provider* (Bagian 2) — Operasi Mutasi (`POST`, `PUT` vs `PATCH`, `DELETE`), Transaksi KRS ACID, & Otentikasi Bearer Token Sanctum (45 Menit)

Kini kita masuk ke inti rekayasa REST API sesungguhnya: memanipulasi data secara semantik, menjaga konsistensi kuota dengan transaksi ACID, menangani error validasi secara global, serta mengunci rute mutasi dengan **Stateless Bearer Token**.

Langkah 4.1: Mengonfigurasi Global JSON Exception Handler di `bootstrap/app.php`

Apa jadinya jika pengguna mengirim input yang tidak lolos `$request->validate()` atau mengirim request tanpa Bearer Token ke rute `/api/*`? Agar Laravel **selalu** mengembalikan format **RFC 7807 (`application/problem+json`)** secara konsisten, perbarui berkas **`bootstrap/app.php`** Anda menjadi seperti berikut:

PHP

```
<?php

use Illuminate\Auth\AuthenticationException;
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use Illuminate\Http\Request;
use Illuminate\Validation\ValidationException;
use Symfony\Component\HttpKernel\Exception\NotFoundHttpException;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
```

```
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        $middleware->validateCsrfTokens(except: [
            'soap/*',
        ]);
    })
    ->withExceptions(function (Exceptions $exceptions) {
        // 1. Paksa setiap rute berawalan /api/* merender respons JSON (bukan halaman HTML!)
        $exceptions->shouldRenderJsonWhen(function (Request $request, Throwable $e) {
            return $request->is('api/*') || $request->expectsJson();
        });

        // 2. Standarkan Error Validasi Input (422 Unprocessable Content - RFC 7807)
        $exceptions->render(function (ValidationException $e, Request $request) {
            if ($request->is('api/*')) {
                return response()->json([
                    'type'     => url('/docs/errors/validation-failed'),
                    'title'    => 'Unprocessable Content',
                    'status'   => 422,
                    'detail'   => 'Satu atau lebih atribut input yang dikirim tidak memenuhi aturan validasi.',
                    'instance' => $request->getRequestUri(),
                    'errors'   => $e->errors(),
                ], 422)->header('Content-Type', 'application/problem+json');
            }
        });

        // 3. Standarkan Error Belum Login / Token Tidak Valid (401 Unauthorized - RFC 7807)
        $exceptions->render(function (AuthenticationException $e, Request $request) {
            if ($request->is('api/*')) {
                return response()->json([
                    'type'     => url('/docs/errors/unauthenticated'),
                    'title'    => 'Unauthorized',
                    'status'   => 401,
                    'detail'   => 'Akses ditolak. Sertakan Bearer Token yang valid pada header Authorization.',
                    'instance' => $request->getRequestUri(),
                ], 401)->header('Content-Type', 'application/problem+json');
            }
        });

        // 4. Standarkan Error Endpoint Salah Ketik (404 Not Found - RFC 7807)
        $exceptions->render(function (NotFoundHttpException $e, Request $request) {
            if ($request->is('api/*')) {
                return response()->json([
                    'type'     => url('/docs/errors/endpoint-not-found'),
                    'title'    => 'Not Found',
                    'status'   => 404,
                    'detail'   => 'Alamat URI endpoint atau resource yang diminta tidak ditemukan.',
                    'instance' => $request->getRequestUri(),
                ], 404)->header('Content-Type', 'application/problem+json');
            }
        });
    })->create();
```

Langkah 4.2: Menambahkan Method `store`, `update` (`PUT` vs `PATCH`), & `destroy` pada `CourseApiController`

Buka kembali **`app/Http/Controllers/Api/V1/CourseApiController.php`**, lalu tambahkan ketiga method berikut tepat di bawah method `show()`:

PHP

```
    /**
     * POST /api/v1/courses
     * Membuat mata kuliah baru -> Wajib mereturn 201 Created + Header Location
     */
    public function store(Request $request): JsonResponse
    {
        $validated = $request->validate([
            'code'        => 'required|string|max:10|unique:courses,code',
            'name'        => 'required|string|max:150',
```

```
            'credits'     => 'required|integer|min:1|max:6',
            'semester'    => 'required|string|max:30',
            'quota'       => 'required|integer|min:0|max:200',
            'description' => 'nullable|string|max:1000',
        ]);

        $validated['code'] = strtoupper(trim($validated['code']));
        $course = Course::create($validated);
        $course->loadCount('enrollments');

        return $this->createdResponse(
            data: new CourseResource($course),
            locationUrl: url("/api/v1/courses/{$course->id}"),
            message: 'Mata kuliah baru berhasil dibuat.'
        );
    }

    /**
     * PUT / PATCH /api/v1/courses/{id}
     * Mendemonstrasikan perbedaan semantik PUT (Full Replace) vs PATCH (Partial Update)
     */
    public function update(Request $request, string $id): JsonResponse
    {
        $course = Course::find($id);

        if (!$course) {
            return $this->problemResponse(
                title: 'Resource Not Found',
                detail: "Mata kuliah dengan ID '{$id}' tidak ditemukan.",
                status: 404,
                errorSlug: 'course-not-found'
            );
        }

        $isPut = $request->isMethod('put');

        if ($isPut) {
            // SEMANTIK PUT: Seluruh field representasi WAJIB dikirimkan secara utuh!
            $validated = $request->validate([
                'code'        => "required|string|max:10|unique:courses,code,{$course->id}",
                'name'        => 'required|string|max:150',
                'credits'     => 'required|integer|min:1|max:6',
                'semester'    => 'required|string|max:30',
                'quota'       => 'required|integer|min:0|max:200',
                'description' => 'nullable|string|max:1000',
            ]);
            // Pada PUT murni, jika field nullable (description) tidak dikirim di body, ia direset ke null
            $validated['description'] = $request->input('description', null);
            $validated['code']        = strtoupper(trim($validated['code']));
        } else {
            // SEMANTIK PATCH: HANYA memvalidasi & mengubah atribut yang dikirim di body saja!
            $validated = $request->validate([
                'code'        => "sometimes|required|string|max:10|unique:courses,code,{$course->id}",
                'name'        => 'sometimes|required|string|max:150',
                'credits'     => 'sometimes|required|integer|min:1|max:6',
                'semester'    => 'sometimes|required|string|max:30',
                'quota'       => 'sometimes|required|integer|min:0|max:200',
                'description' => 'sometimes|nullable|string|max:1000',
            ]);

            if (empty($validated)) {
                return $this->problemResponse(
                    title: 'Bad Request',
                    detail: 'Permintaan PATCH wajib menyertakan minimal satu atribut yang ingin diperbarui.',
                    status: 400,
                    errorSlug: 'empty-patch-payload'
                );
            }

            if (isset($validated['code'])) {
                $validated['code'] = strtoupper(trim($validated['code']));
            }
        }
```

```
        $course->update($validated);
        $course->loadCount('enrollments');

        return $this->successResponse(
            data: new CourseResource($course),
            message: $isPut
                ? 'Seluruh representasi mata kuliah berhasil diganti (PUT).'
                : 'Atribut mata kuliah berhasil diperbarui sebagian (PATCH).'
        );
    }

    /**
     * DELETE /api/v1/courses/{id}
     * Menghapus mata kuliah -> 409 Conflict jika masih punya peserta KRS, 204 No Content jika sukses
     */
    public function destroy(string $id): JsonResponse
    {
        $course = Course::withCount('enrollments')->find($id);

        if (!$course) {
            return $this->problemResponse(
                title: 'Resource Not Found',
                detail: "Mata kuliah dengan ID '{$id}' tidak ditemukan.",
                status: 404,
                errorSlug: 'course-not-found'
            );
        }

        // Aturan Bisnis: Tolak penghapusan dengan 409 Conflict jika sudah ada mahasiswa terdaftar!
        if ($course->enrollments_count > 0) {
            return $this->problemResponse(
                title: 'Conflict – Course Has Active Enrollments',
                detail: "Mata kuliah '{$course->code}' tidak dapat dihapus karena sedang diambil oleh {$course-
>enrollments_count} mahasiswa.",
                status: 409,
                errorSlug: 'course-has-enrollments'
            );
        }

        $course->delete();

        return $this->noContentResponse();
    }
```

Langkah 4.3: Membangun Sub-Resource & Transaksi KRS ACID via API (`CourseEnrollmentApiController`)

Ingat bagaimana di **Minggu 6** kita membangun fitur pengambilan & pembatalan KRS menggunakan `DB::transaction()` dan `lockForUpdate()`? Sekarang mari kita buktikan keindahan arsitektur berlapis: logika transaksi ACID yang persis sama dapat diekspos menjadi endpoint RESTful bagi aplikasi Mobile Mahasiswa!

Buat controller baru:

BASH

```
php artisan make:controller Api/V1/CourseEnrollmentApiController
```

Buka `app/Http/Controllers/Api/V1/CourseEnrollmentApiController.php` dan isi dengan kode berikut:

PHP

```
<?php

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Http\Resources\Api\V1\EnrollmentResource;
use App\Models\Course;
use App\Models\Enrollment;
use App\Traits\ApiResponse;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
```

```
use Illuminate\Support\Facades\DB;

class CourseEnrollmentApiController extends Controller
{
    use ApiResponse;

    /**
     * GET /api/v1/courses/{courseId}/enrollments
     * Menampilkan daftar peserta pada satu mata kuliah (Nested Sub-Resource)
     */
    public function index(string $courseId): JsonResponse
    {
        $course = Course::with('enrollments')->find($courseId);

        if (!$course) {
            return $this->problemResponse('Resource Not Found', 'Mata kuliah tidak ditemukan.', 404, 'course-not-
found');
        }

        return $this->successResponse(
            data: EnrollmentResource::collection($course->enrollments),
            message: "Daftar mahasiswa peserta mata kuliah {$course->code} berhasil diambil.",
            meta: [
                'course_code'     => $course->code,
                'course_name'     => $course->name,
                'total_enrolled'  => $course->enrollments->count(),
                'remaining_quota' => $course->quota,
            ]
        );
    }

    /**
     * POST /api/v1/courses/{courseId}/enrollments
     * Mendaftarkan mahasiswa ke mata kuliah dengan jaminan ACID + Pessimistic Locking (Materi W6)
     */
    public function store(Request $request, string $courseId): JsonResponse
    {
        $validated = $request->validate([
            'student_nim'  => 'required|string|max:15',
            'student_name' => 'required|string|max:150',
            'semester'     => 'nullable|string|max:30',
        ]);

        return DB::transaction(function () use ($validated, $courseId) {
            // 1. Kunci baris mata kuliah secara eksklusif (SELECT ... FOR UPDATE) untuk menangkal Race Condition
            $course = Course::where('id', $courseId)->lockForUpdate()->first();

            if (!$course) {
                return $this->problemResponse('Resource Not Found', 'Mata kuliah tidak ditemukan.', 404, 'course-not-
found');
            }

            // 2. Cek apakah mahasiswa sudah terdaftar di mata kuliah ini -> 409 Conflict
            $alreadyExists = Enrollment::where('course_id', $course->id)
                ->where('student_nim', $validated['student_nim'])
                ->exists();

            if ($alreadyExists) {
                return $this->problemResponse(
                    title: 'Duplicate Enrollment Conflict',
                    detail: "Mahasiswa dengan NIM '{$validated['student_nim']}' sudah terdaftar pada mata kuliah
{$course->code}.",
                    status: 409,
                    errorSlug: 'duplicate-enrollment'
                );
            }

            // 3. Cek ketersediaan sisa kuota kursi -> 422 Unprocessable Content jika penuh
            if ($course->quota < 1) {
                return $this->problemResponse(
                    title: 'Course Quota Full',
                    detail: "Maaf, kuota kursi untuk mata kuliah {$course->code} telah habis (0 tersisa).",
                    status: 422,
                    errorSlug: 'quota-exhausted'
```

```
                );
            }

            // 4. Kurangi kuota (-1) dan catat baris KRS secara atomik
            $course->decrement('quota');

            $enrollment = $course->enrollments()->create([
                'student_nim'  => strtoupper(trim($validated['student_nim'])),
                'student_name' => $validated['student_name'],
                'semester'     => $validated['semester'] ?? $course->semester,
            ]);

            return $this->createdResponse(
                data: new EnrollmentResource($enrollment),
                locationUrl: url("/api/v1/courses/{$course->id}/enrollments"),
                message: "Berhasil mengambil KRS mata kuliah {$course->code}. Sisa kuota kini: {$course->fresh()->quota}."
            );
        });
    }

    /**
     * DELETE /api/v1/enrollments/{enrollmentId}
     * Membatalkan KRS (Drop Course) dan mengembalikan kuota (+1) secara atomik
     */
    public function destroy(string $enrollmentId): JsonResponse
    {
        return DB::transaction(function () use ($enrollmentId) {
            $enrollment = Enrollment::find($enrollmentId);

            if (!$enrollment) {
                return $this->problemResponse('Resource Not Found', 'Data pendaftaran KRS tidak ditemukan.', 404, 'enrollment-not-found');
            }

            $course = Course::where('id', $enrollment->course_id)->lockForUpdate()->first();
            $enrollment->delete();

            if ($course) {
                $course->increment('quota');
            }

            return $this->noContentResponse();
        });
    }
}
```

Langkah 4.4: Membangun Otentikasi Stateless Bearer Token (`AuthTokenController`)

Agar tidak sembarang orang bisa menghapus atau menambah mata kuliah di API kita, mari aktifkan **Bearer Token Authentication** menggunakan **Laravel Sanctum**!

1. **Tambahkan Trait `HasApiTokens` pada Model `User`**: Buka `app/Models/User.php`, pastikan `Laravel\Sanctum\HasApiTokens` di-import dan dipasang di dalam class `User`:

PHP

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens; // <--- 1. Pastikan baris import ini ada!

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable; // <--- 2. Pasang HasApiTokens di sini!
    // ... kode bawaan lainnya tetap sama ...
}
```

2. Buat `AuthTokenController`:

BASH

```
php artisan make:controller Api/V1/AuthTokenController
```

Buka **`app/Http/Controllers/Api/V1/AuthTokenController.php`** dan isi dengan kode ringkas berikut:

PHP

```
<?php

namespace App\Http\Controllers\Api\V1;

use App\Http\Controllers\Controller;
use App\Models\User;
use App\Traits\ApiResponse;
use Illuminate\Http\JsonResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

class AuthTokenController extends Controller
{
    use ApiResponse;

    /**
     * POST /api/v1/auth/login
     * Memverifikasi kredensial & menerbitkan Stateless Bearer Token baru
     */
    public function login(Request $request): JsonResponse
    {
        $validated = $request->validate([
            'email'       => 'required|email',
            'password'    => 'required|string',
            'device_name' => 'nullable|string|max:100',
        ]);

        $user = User::where('email', $validated['email'])->first();

        if (!$user || !Hash::check($validated['password'], $user->password)) {
            return $this->problemResponse(
                title: 'Invalid Credentials',
                detail: 'Kombinasi email atau password yang diberikan tidak cocok.',
                status: 401,
                errorSlug: 'invalid-credentials'
            );
        }

        $deviceName = $validated['device_name'] ?? 'postman-client';
        $token      = $user->createToken($deviceName)->plainTextToken;

        return $this->successResponse(
            data: [
                'token_type'   => 'Bearer',
                'access_token' => $token,
                'user'         => [
                    'id'    => $user->id,
                    'name'  => $user->name,
                    'email' => $user->email,
                ],
            ],
            message: 'Otentikasi API berhasil. Lampirkan access_token pada header Authorization: Bearer <token>.'
        );
    }

    /**
     * GET /api/v1/auth/me (Membutuhkan Bearer Token)
     */
    public function me(Request $request): JsonResponse
    {
        return $this->successResponse(
            data: $request->user(),
            message: 'Profil pemilik token aktif berhasil diambil.'
        );
```

```
    }

    /**
     * POST /api/v1/auth/logout (Mencabut / Revoke token yang sedang dipakai)
     */
    public function logout(Request $request): JsonResponse
    {
        $request->user()->currentAccessToken()->delete();

        return $this->successResponse(
            data: null,
            message: 'Bearer Token berhasil dicabut (revoked). Sesi API diakhiri.'
        );
    }
}
```

Langkah 4.5: Menyusun Peta Rute Lengkap pada `routes/api.php`

Sekarang buka berkas `routes/api.php` dan perbarui secara utuh agar memisahkan **Endpoint Publik (Read-Only)** dan **Endpoint Terproteksi Token (`auth:sanctum`)**:

PHP

```php
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\Api\V1\AuthTokenController;
use App\Http\Controllers\Api\V1\CourseApiController;
use App\Http\Controllers\Api\V1\CourseEnrollmentApiController;
use App\Http\Controllers\Api\V1\GithubPortfolioController;

Route::prefix('v1')->group(function () {

    // ========================================================================
    // 1. ENDPOINT PUBLIK (Dapat diakses tanpa token: Login & Katalog Baca)
    // ========================================================================
    Route::post('/auth/login', [AuthTokenController::class, 'login']);

    Route::get('/portfolios/github/{username}', [GithubPortfolioController::class, 'show']);

    Route::get('/courses', [CourseApiController::class, 'index']);
    Route::get('/courses/{identifier}', [CourseApiController::class, 'show']);
    Route::get('/courses/{courseId}/enrollments', [CourseEnrollmentApiController::class, 'index']);

    // ========================================================================
    // 2. ENDPOINT TERPROTEKSI STATELESS BEARER TOKEN (Wajib auth:sanctum)
    // ========================================================================
    Route::middleware('auth:sanctum')->group(function () {
        Route::get('/auth/me', [AuthTokenController::class, 'me']);
        Route::post('/auth/logout', [AuthTokenController::class, 'logout']);

        // Mutasi Katalog Mata Kuliah
        Route::post('/courses', [CourseApiController::class, 'store']);
        Route::put('/courses/{id}', [CourseApiController::class, 'update']);
        Route::patch('/courses/{id}', [CourseApiController::class, 'update']);
        Route::delete('/courses/{id}', [CourseApiController::class, 'destroy']);

        // Transaksi Pengambilan & Pembatalan KRS
        Route::post('/courses/{courseId}/enrollments', [CourseEnrollmentApiController::class, 'store']);
        Route::delete('/enrollments/{enrollmentId}', [CourseEnrollmentApiController::class, 'destroy']);
    });
});
```

Verifikasi seluruh rute Anda telah terdaftar rapi menggunakan perintah Artisan:

BASH

```bash
php artisan route:list --path=api/v1
```

LAB 5: Kontrak OpenAPI 3.0 (`openapi.yaml`), Pengujian Sistematis di Postman, & Troubleshooting (25 Menit)

Langkah 5.1: Menyusun Kontrak Spesifikasi Layanan (`docs/openapi.yaml`)

Buat folder `docs` di akar proyek `sim-del`, lalu buat berkas `docs/openapi.yaml`. Berkas ini adalah **Service Contract** modern pengganti WSDL pada dunia REST API:

YAML

```yaml
openapi: 3.0.3
info:
  title: SIM-Del Academic RESTful Web API
  version: 1.0.0
  description: >
    Kontrak resmi layanan REST API Sistem Informasi Akademik IT Del (Minggu 9 PPW).
    Mematuhi Richardson Maturity Model Level 3 (HATEOAS) & RFC 7807 Problem Details.

servers:
  - url: http://localhost:8000/api/v1
    description: Local Development Server

components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      description: Masukkan token hasil login dari endpoint POST /auth/login

  schemas:
    CourseInput:
      type: object
      required: [code, name, credits, semester, quota]
      properties:
        code: { type: string, example: "12S3109" }
        name: { type: string, example: "Arsitektur Berorientasi Layanan" }
        credits: { type: integer, minimum: 1, maximum: 6, example: 3 }
        semester: { type: string, example: "2025/2026-Genap" }
        quota: { type: integer, minimum: 0, example: 40 }
        description: { type: string, nullable: true, example: "Mata kuliah pilihan SOA & Microservices" }

    ProblemDetails:
      type: object
      properties:
        type: { type: string, example: "http://localhost:8000/docs/errors/validation-failed" }
        title: { type: string, example: "Unprocessable Content" }
        status: { type: integer, example: 422 }
        detail: { type: string }
        instance: { type: string, example: "/api/v1/courses" }

paths:
  /auth/login:
    post:
      summary: Menerbitkan Stateless Bearer Token
      responses:
        '200': { description: Login berhasil & mengembalikan access_token }
        '401': { description: Kredensial tidak valid }

  /courses:
    get:
      summary: Daftar mata kuliah (mendukung search, filter, sort, & pagination)
      parameters:
        - { name: search, in: query, schema: { type: string } }
        - { name: credits, in: query, schema: { type: integer } }
        - { name: available, in: query, schema: { type: boolean } }
        - { name: per_page, in: query, schema: { type: integer, default: 5 } }
      responses:
        '200': { description: Koleksi mata kuliah terpaginasi }
    post:
      summary: Menambah mata kuliah baru
      security:
        - BearerAuth: []
      requestBody:
        required: true
```

```
      content:
        application/json:
          schema: { $ref: '#/components/schemas/CourseInput' }
    responses:
      '201': { description: Mata kuliah berhasil dibuat (beserta Header Location) }
      '401': { description: Belum melampirkan Bearer Token }
      '422':
        description: Gagal validasi input
        content:
          application/problem+json:
            schema: { $ref: '#/components/schemas/ProblemDetails' }

/courses/{id}:
  get:
    summary: Detail satu mata kuliah beserta peserta KRS & link HATEOAS
    parameters:
      - { name: id, in: path, required: true, schema: { type: string } }
    responses:
      '200': { description: Detail ditemukan }
      '404': { description: Mata kuliah tidak ditemukan }
  put:
    summary: Mengganti seluruh atribut mata kuliah (Full Replacement)
    security: [{ BearerAuth: [] }]
    parameters:
      - { name: id, in: path, required: true, schema: { type: integer } }
    responses:
      '200': { description: Seluruh data diganti }
  patch:
    summary: Mengubah sebagian atribut mata kuliah (Partial Update)
    security: [{ BearerAuth: [] }]
    parameters:
      - { name: id, in: path, required: true, schema: { type: integer } }
    responses:
      '200': { description: Sebagian atribut diperbarui }
  delete:
    summary: Menghapus mata kuliah (ditolak 409 jika sudah ada peserta KRS)
    security: [{ BearerAuth: [] }]
    parameters:
      - { name: id, in: path, required: true, schema: { type: integer } }
    responses:
      '204': { description: Berhasil dihapus tanpa body }
      '409': { description: Konflik — masih ada mahasiswa mengambil MK ini }
```

*(Anda dapat menyalin isi `docs/openapi.yaml` di atas ke laman **https://editor.swagger.io** untuk melihat dokumentasi interaktif layaknya dokumentasi resmi GitHub/Stripe!)*

Langkah 5.2: Pengujian Sistematis 12 Skenario di Postman (Wajib Dicoba Semua!)

Agar pengujian di Postman sangat praktis tanpa menyalin URL dan Token berulang kali:

1. Buat **Collection** baru di Postman bernama `PPW Week 9 - SIM-Del Web Services`.
2. Pada tab **Variables** milik Collection tersebut, tambahkan 2 variabel:
   - `base_url` = `http://localhost:8000/api/v1`
   - `token` = *(kosongkan dulu, nanti diisi nilai `access_token` setelah menjalankan skenario #3)*
3. Pastikan Anda memiliki minimal 1 akun User di tabel `users` (contoh lewat Tinker jika belum ada: `php artisan tinker --execute="App\Models\User::updateOrCreate(['email'=>'akademik@del.ac.id'], ['name'=>'Admin Akademik','password'=>bcrypt('password123')]);"`).
4. Jalankan **12 Skenario Pengujian** pada tabel di bawah ini secara berurutan untuk membuktikan pemahaman menyeluruh Anda atas SOAP & REST:

No	Metode & Endpoint Uji	Konfigurasi Body / Auth	Kode Status Ekspektasi	Hal Penting yang Wajib Anda Amati
1	`POST /soap/academic`	Body XML `<nim>12S22001</nim>`	`200 OK`	Respons XML `<soap:Envelope>` berisi `<m:total_sks>`.
2	`POST /soap/academic`	Body XML `<nim>00000</nim>`	`500 Error`	Respons XML `<soap:Fault>` dengan `<faultstring>`.

No	Metode & Endpoint Uji	Konfigurasi Body / Auth	Kode Status Ekspektasi	Hal Penting yang Wajib Anda Amati
3	`POST {{base_url}}/auth/login`	JSON `{"email":"akademik@del.ac.id","password":"password123"}`	**`200 OK`**	Salin nilai `data.access_token` (`1|...`) ke variabel `{{token}}`!
4	`POST {{base_url}}/courses`	**Tanpa Token** (No Auth)	**`401 Unauthorized`**	Ditolak dalam format RFC 7807 `application/problem+json`.
5	`POST {{base_url}}/courses`	Bearer `{{token}}` + JSON kosong `{}`	**`422 Unprocessable`**	Muncul rincian kesalahan per kolom di objek `errors`.
6	`POST {{base_url}}/courses`	Bearer `{{token}}` + JSON MK `12S3109` (`quota: 1`, `description: "Kelas A"`)	**`201 Created`**	Cek tab **Headers** Response: terdapat header `Location` menuju ID baru!
7	`PATCH {{base_url}}/courses/{id}`	Bearer `{{token}}` + JSON **`{"quota": 2}` saja!**	**`200 OK`**	`remaining_quota` berubah jadi `2`, sedangkan `name` & `description` **tetap utuh**!
8	`PUT {{base_url}}/courses/{id}`	Bearer `{{token}}` + JSON lengkap **tanpa** menyertakan `description`	**`200 OK`**	Seluruh data terganti dan `description` direset menjadi **`null`** (Bukti nyata `PUT` vs `PATCH`!).
9	`POST {{base_url}}/courses/{id}/enrollments`	Bearer `{{token}}` + JSON `{"student_nim":"12S22005","student_name":"Siti"}`	**`201 Created`**	Kuota berkurang otomatis secara atomik (`DB::transaction`).
10	`POST {{base_url}}/courses/{id}/enrollments`	Kirim ulang JSON NIM `12S22005` yang sama persis	**`409 Conflict`**	Ditolak karena duplikasi pendaftaran KRS; kuota **tidak ikut berkurang**!
11	`DELETE {{base_url}}/courses/{id}`	Bearer `{{token}}` (saat MK masih punya peserta Siti)	**`409 Conflict`**	Ditolak demi menjaga integritas data pendaftaran aktif!
12	`DELETE {{base_url}}/enrollments/{id}` lalu `DELETE {{base_url}}/courses/{id}`	Bearer `{{token}}`	**`204 No Content`**	Body respons benar-benar kosong (`0 B`) dengan status hijau `204 No Content`!

Langkah 5.3: Kamus Troubleshooting Masalah Umum Praktikum REST API Laravel 11

Jika Anda menemui kendala saat praktikum atau mengerjakan tugas mandiri, periksa tabel diagnosis cepat berikut:

Gejala Error yang Muncul	Penyebab Akar Masalah	Solusi Perbaikan Cepat
Saat mengirim input salah di Postman, bukan muncul JSON `422` melainkan tampil **Halaman HTML Welcome Laravel (`200` / `302`)**	Anda lupa menyertakan header `Accept: application/json`, atau blok `shouldRenderJsonWhen` di `bootstrap/app.php` belum dipasang.	Tambahkan Header `Accept: application/json` di Postman dan pastikan Langkah 4.1 pada `bootstrap/app.php` sudah disimpan.
Muncul error **`419 Page Expired`** saat melakukan `POST` / `PATCH` / `DELETE`	Anda salah meletakkan rute API di dalam `routes/web.php` (yang memiliki proteksi CSRF form browser).	Pindahkan rute API ke dalam berkas **`routes/api.php`** (sehingga diakses lewat `/api/v1/...`).
Muncul error `500`: **`Call to undefined method App\Models\User::createToken()`**	Model `app/Models/User.php` belum dipasangi trait `Laravel\Sanctum\HasApiTokens`.	Tambahkan `use Laravel\Sanctum\HasApiTokens;` dan pasang `use HasApiTokens;` di dalam class `User` (lihat Langkah 4.4).
Muncul error `500`: **`Add [code] to fillable property to allow mass assignment`**	Kolom tabel belum didaftarkan ke dalam array `protected $fillable = [...]` pada Model Eloquent.	Buka `app/Models/Course.php` (atau `Buku.php` / `Peminjaman.php`) dan lengkapi array `$fillable`.

Gejala Error yang Muncul	Penyebab Akar Masalah	Solusi Perbaikan Cepat
Saat memanggil GitHub API dari `Http::get()` di Windows Laragon/XAMPP muncul **`cURL error 60: SSL certificate problem`**	PHP lokal di Windows belum memiliki berkas bundel sertifikat `cacert.pem` yang terkonfigurasi di `php.ini`.	Unduh `https://curl.se/ca/cacert.pem` ke folder PHP lalu set `curl.cainfo = "C:\...\cacert.pem"` di `php.ini`, atau khusus lokal dev gunakan `Http::withoutVerifying()` secara sementara.
Perintah `curl -H ...` di terminal Windows PowerShell menghasilkan error **`Cannot bind parameter 'Headers'`**	Di PowerShell, kata `curl` adalah alias milik `Invoke-WebRequest`, bukan program cURL asli.	Ketikkan **`curl.exe`** (dengan `.exe`) atau jalankan perintah di terminal **Git Bash / CMD**.

V. TUGAS MANDIRI MAHASISWA (CONTINUATION ASSIGNMENT DARI MINGGU 6)

5.1 Judul Penugasan Mandiri Lanjutan

> **Pengembangan Lanjutan Sistem Informasi Perpustakaan Kampus Del (`SIPUS-Del` Tahap 3): Analisis Kelayakan Layanan, Simulasi Ringkas Endpoint SOAP Legacy, serta Implementasi Mendalam RESTful Web API Provider & Consumer (API v1) Berbasis Eloquent Resource, Transaksi Sirkulasi ACID, Stateless Bearer Token, HATEOAS, & Kontrak OpenAPI 3.0 pada Laravel 11.x**

5.2 Latar Belakang & Skenario Kebutuhan Integrasi Sistem

Pada **Tugas Mandiri Minggu 6**, Anda telah membangun aplikasi web monolitik **`SIPUS-Del`** yang memiliki tabel **`kategoris`**, **`bukus`** (beserta kolom `stok`), **`peminjamans`**, dan **`users`**.

Kini pada **Minggu 9**, UPT Perpustakaan IT Del menghadapi tiga kebutuhan integrasi lintas sistem:

1. **Integrasi Sistem Bebas Pustaka Yudisium (Sistem Legacy SOAP)**: Aplikasi Administrasi Yudisium kampus versi lama masih berbasis SOAP XML dan membutuhkan satu endpoint sederhana untuk mengecek apakah seorang mahasiswa (`nim_peminjam`) masih memiliki tanggungan buku yang belum dikembalikan.
2. **Peluncuran Aplikasi Mobile & Kiosk Mandiri Perpustakaan (REST API Provider)**: Perpustakaan membutuhkan layanan **RESTful API (`/api/v1/...`)** yang cepat, ringan, dan *stateless* untuk menampilkan katalog buku serta memproses transaksi peminjaman dan pengembalian buku dengan aman.
3. **Pengayaan Metadata Buku Eksternal (REST API Consumer)**: Saat menampilkan detail buku berdasarkan ISBN, sistem dapat mengambil informasi tambahan (seperti jumlah halaman, penerbit, atau tautan pratinjau) dari layanan publik eksternal **OpenLibrary REST API** (`https://openlibrary.org/api/books?bibkeys=ISBN:{isbn}&format=json&jscmd=data`) menggunakan fasad `Http` Laravel.

5.3 Spesifikasi Teknis Wajib (Requirements Checklist)

Lanjutkan kode proyek **`SIPUS-Del`** Minggu 6 Anda dengan mengimplementasikan 5 spesifikasi terukur berikut (antarmuka web Blade Minggu 6 tetap dipertahankan dan tidak boleh rusak):

1. Analisis Kelayakan (*Feasibility*) & Endpoint Ringkas SOAP Bebas Pustaka (Bobot: 15%)

- Menyusun bagian **Analisis Kebutuhan & Kelayakan Web Services** pada `README.md` (menjelaskan peran *Provider* vs *Consumer* pada `SIPUS-Del`, serta alasan teknis pemilihan REST untuk Mobile/Kiosk dibanding SOAP).
- Membuat `Soap/LibrarySoapController` dengan 2 rute:
  - `GET /soap/sipus.wsdl`: Menampilkan kontrak XML WSDL ringkas untuk operasi `CheckClearanceStatus`.
  - `POST /soap/sipus`: Menerima `<soap:Envelope>` berisi `<nim_peminjam>`, menghitung jumlah buku berstatus `'dipinjam'` milik NIM tersebut di tabel `peminjamans`, lalu mengembalikan XML `<soap:Envelope>` berisi `<active_loans>` dan

`<is_cleared>` (`true` jika `0` pinjaman aktif, `false` jika masih ada pinjaman). Kembalikan `<soap:Fault>` jika tag `<nim_peminjam>` kosong atau XML cacat.

2. Implementasi REST API Provider Katalog Kategori & Buku (`/api/v1/bukus`) (Bobot: 25%)

- Mengaktifkan `php artisan install:api` dan membuat **`BukuResource`** (`JsonResource`) serta trait/helper respons JSON yang seragam.
- Mengimplementasikan **`Api/V1/BukuApiController`** dengan pemetaan semantik penuh (RMM Level 2 & 3):
  - **`GET /api/v1/bukus`** *(Publik)*: Menampilkan koleksi buku terpaginasi lengkap dengan blok `meta`, mendukung pencarian `?search=...` (judul/penulis/ISBN), filter `?kategori_id=...`, filter ketersediaan `?tersedia=1` (`stok > 0`), serta pengurutan `?sort=...`. Wajib bebas dari *N+1 Query Problem* (`with('kategori')->withCount('peminjamans')`).
  - **`GET /api/v1/bukus/{id}`** *(Publik)*: Menampilkan detail satu buku beserta objek kategori dan daftar riwayat peminjamannya (atau status **`404 Not Found`** berformat RFC 7807 jika tidak ditemukan).
  - **`POST /api/v1/bukus`** *(Wajib Bearer Token)*: Validasi input ketat (ISBN unik, `stok >= 0`, `kategori_id` valid di tabel `kategoris`), menyimpan buku baru, dan mengembalikan status **`201 Created`** beserta header **`Location: /api/v1/bukus/{id}`**.
  - **`PUT /api/v1/bukus/{id}`** vs **`PATCH /api/v1/bukus/{id}`** *(Wajib Bearer Token)*: Membuktikan perbedaan nyata antara penggantian penuh (`PUT` — seluruh atribut wajib dikirim) dan modifikasi parsial (`PATCH` — menggunakan `sometimes|required`, misalnya hanya memperbarui `stok` buku).
  - **`DELETE /api/v1/bukus/{id}`** *(Wajib Bearer Token)*: Menolak penghapusan dengan kode **`409 Conflict`** jika buku sedang memiliki peminjaman aktif (`status = 'dipinjam'`), dan mengembalikan **`204 No Content`** jika berhasil dihapus.

3. REST API Transaksi Sirkulasi Peminjaman ACID & Otentikasi Bearer Token Sanctum (Bobot: 30%)

- **Otentikasi Stateless API (`/api/v1/auth/...`)**:
  - `POST /api/v1/auth/login`: Memverifikasi email & password pustakawan dan menerbitkan Sanctum Bearer Token (`plainTextToken`).
  - `GET /api/v1/auth/me` & `POST /api/v1/auth/logout`: Mengecek profil pemilik token dan mencabut (*revoke*) token aktif (`auth:sanctum`).
- **Transaksi Sirkulasi RESTful (`Api/V1/PeminjamanApiController`)**:
  - **`GET /api/v1/peminjamans`** *(Wajib Bearer Token)*: Menampilkan daftar transaksi peminjaman terpaginasi dengan filter `?status=dipinjam|dikembalikan` dan `?nim=...`.
  - **`POST /api/v1/peminjamans`** *(Wajib Bearer Token)*: Mencatat peminjaman buku baru via JSON (`buku_id`, `nim_peminjam`, `nama_peminjam`, `durasi_hari`). **Wajib dibungkus `DB::transaction()` dengan `lockForUpdate()` pada baris buku**:
  - Jika `stok < 1`, batalkan dan kembalikan **`422 Unprocessable Content`** ("*Stok buku sedang habis*").
  - Jika mahasiswa masih meminjam buku yang sama dan belum dikembalikan, tolak dengan **`409 Conflict`**.
  - Jika lolos, kurangi `stok` (`-1`), simpan peminjaman baru, dan kembalikan **`201 Created`** + Header `Location`.
  - **`PATCH /api/v1/peminjamans/{id}/kembalikan`** *(Wajib Bearer Token)*: Mengembalikan buku di dalam `DB::transaction()` + `lockForUpdate()`. Jika sudah berstatus `'dikembalikan'`, tolak dengan **`409 Conflict`**. Jika valid, ubah status menjadi `'dikembalikan'`, isi `tanggal_kembali`, tambah `stok` buku (`+1`), dan kembalikan **`200 OK`**.

4. Kontrol Hypermedia HATEOAS, Error Global RFC 7807, & REST Service Consumer (Bobot: 15%)

- **HATEOAS (`_links`) Dinamis**:
  - Pada `BukuResource`: Link `pinjam_buku` hanya muncul di dalam `_links` apabila `stok > 0`.
  - Pada `PeminjamanResource`: Link `kembalikan_buku` (`PATCH`) hanya muncul di dalam `_links` apabila `status === 'dipinjam'`.
- **Global Exception Rendering RFC 7807**: Mengonfigurasi `bootstrap/app.php` sehingga error `401`, `404`, dan `422` pada `/api/*` selalu mereturn struktur `application/problem+json` (`type`, `title`, `status`, `detail`, `instance`).
- **Fitur REST Service Consumer (`Http` Facade)**: Membuat 1 endpoint `GET /api/v1/bukus/{id}/referensi-eksternal` (atau menyertakannya pada detail ISBN) yang memanggil REST API publik eksternal (seperti OpenLibrary API `https://openlibrary.org/isbn/{isbn}.json` atau Google Books API) dengan `Http::timeout(8)->get(...)` beserta penanganan aman jika buku eksternal tidak ditemukan atau koneksi *timeout*.

5. Kontrak Layanan (`openapi-sipus-v1.yaml`), Koleksi Postman, & Dokumentasi Pengujian (Bobot: 15%)

- Menyimpan dokumen kontrak **`docs/openapi-sipus-v1.yaml`** dan ekspor **`docs/SIPUS-Del-API-v1.postman_collection.json`** di dalam repositori.
- Melampirkan *screenshot* bukti pengujian Postman di `README.md` yang mencakup: (a) Uji SOAP XML & `<soap:Fault>`, (b) Login Bearer Token, (c) GET Katalog terpaginasi + HATEOAS, (d) POST Buku `201 Created` + Header `Location`, (e) Bukti perbedaan `PUT` vs `PATCH`, (f) Peminjaman sukses & penolakan stok habis (`422` / `409`), serta (g) Bukti latihan GitHub REST API (dengan token disensor!).

VI. INSTRUKSI PENGELOLAAN GIT & PENGUMPULAN TUGAS

6.1 Strategi Branching & Konvensi Commit (*Conventional Commits*)

Kerjakan penugasan pada branch baru `week9-rest-api` dari repositori `SIPUS-Del` Minggu 6 Anda (atau repositori `ppw-2026-week9-[NIM]`):

BASH

```
git checkout -b week9-rest-api
```

Gunakan riwayat commit yang bertahap, rapi, dan bermakna:

- `feat(soap): add concise LibrarySoapController and WSDL contract for clearance check`
- `feat(api-core): install laravel api routing, ApiResponse trait, and RFC 7807 exception handler`
- `feat(api-auth): implement stateless Sanctum bearer token login and revocation`
- `feat(api-books): build BukuApiController and BukuResource with dynamic HATEOAS links`
- `feat(api-loans): implement ACID book borrowing and return REST endpoints with lockForUpdate`
- `feat(api-consumer): integrate external book metadata lookup via Laravel Http client`
- `docs(contract): add OpenAPI 3.0 specification, Postman collection, and test report in README`

6.2 Disiplin Keamanan Token & Berkas `.env`

- **DILARANG KERAS** melakukan commit berkas `.env` atau menuliskan *Personal Access Token* GitHub (`ghp_...`) secara *hardcode* di dalam kode sumber maupun teks `README.md`! GitHub memiliki pemindai otomatis (*Secret Scanning*) yang akan langsung mencabut token bocor dan menandai repositori Anda berisiko keamanan.
- Saat menampilkan *screenshot* pengujian Postman/cURL di `README.md`, **sensor bagian tengah/akhir token** dan sertakan kredensial akun uji seeder lokal (`pustakawan@del.ac.id` / `password123`) agar dosen dan asisten dapat langsung menguji API Anda.

6.3 Standarisasi Struktur `README.md` Minggu 9

Pastikan `README.md` pada repositori Anda memuat:

1. **Identitas Mahasiswa**: Nama Lengkap, NIM, Kelas, Program Studi.
2. **Analisis Kebutuhan & Kelayakan Web Services**: Matriks peran *Provider/Consumer* pada `SIPUS-Del` serta komparasi ringkas SOAP vs REST untuk studi kasus perpustakaan.
3. **Tabel Katalog Endpoint Web Services (`SOAP` & `REST API v1`)**: Memuat Method, URI, Proteksi Auth, Deskripsi, dan Kode Status Respons.
4. **Panduan Instalasi & Pengujian Cepat**: Perintah `composer install`, `php artisan migrate:fresh --seed`, akun uji pustakawan, dan cara mengimpor koleksi Postman dari folder `docs/`.
5. **Galeri Bukti Pengujian (Screenshots)**: Seluruh 7 skenario pengujian wajib pada poin 5.3.5.

🗓️ **Batas Waktu Pengumpulan**: **Sesuai jadwal pada portal e-Course / CIS Institut Teknologi Del**. Format Tautan Pengumpulan: URL Repositori Publik GitHub `https://github.com/[username]/ppw-2026-week9-[NIM]`.

VII. RUBRIK PENILAIAN PRAKTIKUM ANALITIK

No	Kriteria Evaluasi	Bobot	Sangat Baik (85 - 100)	Baik (70 - 84)	Cukup (55 - 69)	Kurang (< 55)
1	**Analisis Kelayakan & Implementasi Ringkas SOAP/WSDL**	**15%**	Analisis kelayakan tajam dan kontekstual; endpoint WSDL valid; endpoint SOAP mampu memproses `<soap:Envelope>` dari data DB nyata dan mereturn `<soap:Fault>` dengan tepat.	Analisis kelayakan jelas; endpoint WSDL dan SOAP berjalan baik untuk skenario normal.	Endpoint SOAP dibuat namun belum terhubung ke data DB atau belum menangani `<soap:Fault>`.	Tidak mengimplementasikan simulasi SOAP/WSDL dan tidak menyertakan analisis kelayakan.
2	**REST API Provider Katalog Buku (`GET`, `POST`, `PUT` vs `PATCH`, `DELETE`)**	**25%**	Seluruh operasi berjalan mulus di `routes/api.php`; menggunakan `BukuResource` bebas N+1; filter, sort, & paginasi presisi; perbedaan `PUT` vs `PATCH` serta status `201+Location`, `204`, `404`, `409` akurat.	Seluruh operasi CRUD buku berfungsi baik; paginasi dan kode status utama (`200`, `201`, `204`, `404`) terpenuhi.	CRUD berjalan namun belum menggunakan `JsonResource`, belum membedakan `PUT` vs `PATCH`, atau memakai status `200` untuk semua kondisi.	Endpoint tidak mematuhi kaidah URI REST (memakai kata kerja di URL) atau banyak terjadi error `500`.
3	**REST API Sirkulasi Peminjaman ACID & Otentikasi Bearer Token**	**30%**	Login/Logout Sanctum Token berjalan sempurna; peminjaman & pengembalian buku terbungkus `DB::transaction()` + `lockForUpdate()`; penolakan stok habis (`422`) & konflik pinjaman (`409`) presisi tanpa kebocoran stok.	Proteksi `auth:sanctum` dan transaksi peminjaman/pengembalian buku via API berfungsi dengan benar; stok ter-update akurat.	Peminjaman via API berjalan namun belum dikunci dengan transaksi ACID / `lockForUpdate()` atau sebagian rute mutasi belum diproteksi token.	Transaksi peminjaman gagal dijalankan melalui API atau autentikasi masih mengandalkan session web browser.
4	**HATEOAS (`_links`), Error Global RFC 7807, & REST Consumer (`Http`)**	**15%**	Tautan `_links` HATEOAS bersifat kontekstual/dinamis terhadap stok & status pinjam; seluruh galat `/api/*` berformat `application/problem+json` (RFC 7807); konsumsi API luar via `Http` dilengkapi *timeout* & *fallback*.	`_links` HATEOAS tampil rapi, format error JSON konsisten, dan fitur konsumsi API eksternal berjalan.	Hanya sebagian fitur yang diterapkan (misal `_links` statis atau belum menerapkan standar error RFC 7807).	Tidak menerapkan HATEOAS, error API mengembalikan halaman HTML, dan tidak ada konsumsi API eksternal.
5	**Kontrak OpenAPI 3.0, Postman Collection, Bukti Uji, & Git**	**15%**	File `openapi-sipus-v1.yaml` valid; koleksi Postman lengkap dengan variabel; `README.md` memuat 7 bukti uji lengkap; riwayat commit rapi dan bebas dari kebocoran `.env`/token rahasia.	Kontrak OpenAPI, koleksi Postman, dan dokumentasi `README.md` lengkap dan mudah diuji ulang.	Dokumentasi kurang lengkap atau koleksi Postman belum mencakup pengujian skenario negatif (4xx).	Tidak menyertakan kontrak/koleksi Postman, `README.md` kosong, atau membocorkan token rahasia ke publik.